

λ FS: A Scalable and Elastic Distributed File System Metadata Service using Serverless Functions

Benjamin Carver^{*}, Runzhou Han[†], Jingyuan Zhang^{*}, Mai Zheng[†], Yue Cheng[‡]

^{*}George Mason University [†]Iowa State University [‡]University of Virginia

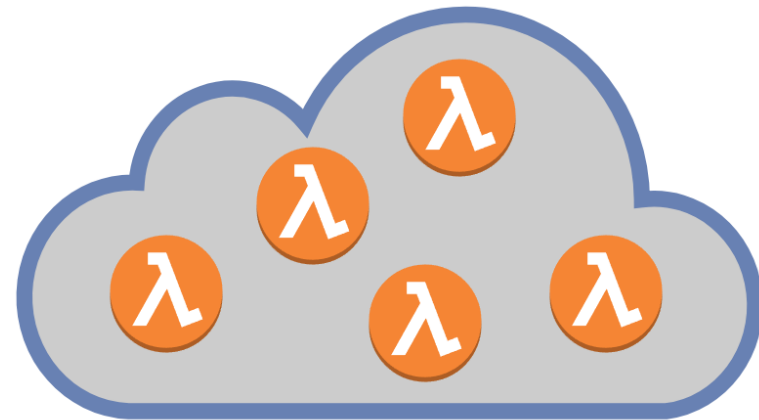
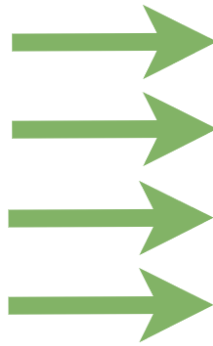


❖ Serverless computing (or FaaS)

- Emerging **functions-as-a-service** model.
- Enables a **new** way of **building & scaling apps & services**.



Monolithic, Server-Based Application



Serverless Functions

Benefits of Serverless Computing

❖ Key Benefit 1:

Elastic Auto-Scaling.

Improved performance & resource efficiency.

❖ Key Benefit 2:

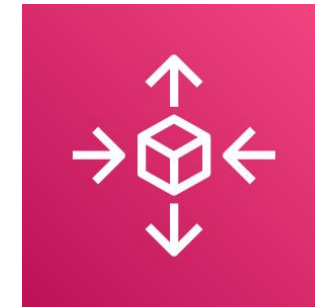
Cost Efficiency.

Pay-Per-Use Pricing.

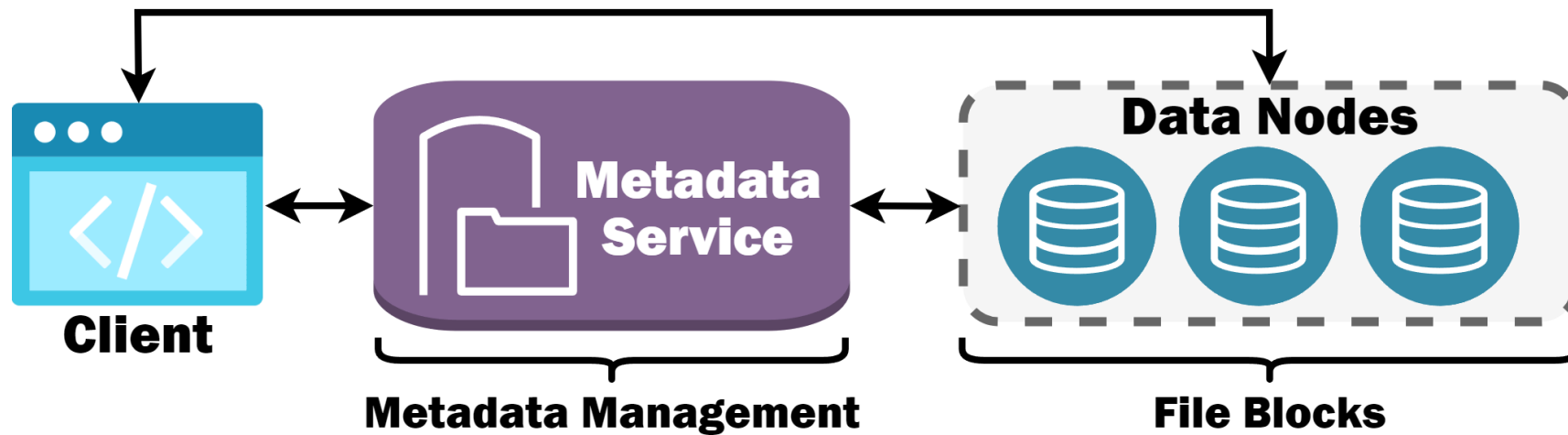
❖ Key Benefit 3:

Automatic Resource Management.

Ease of Use.



The **metadata service (MDS)** sits on the **critical path** for distributed file system (DFS) operations.



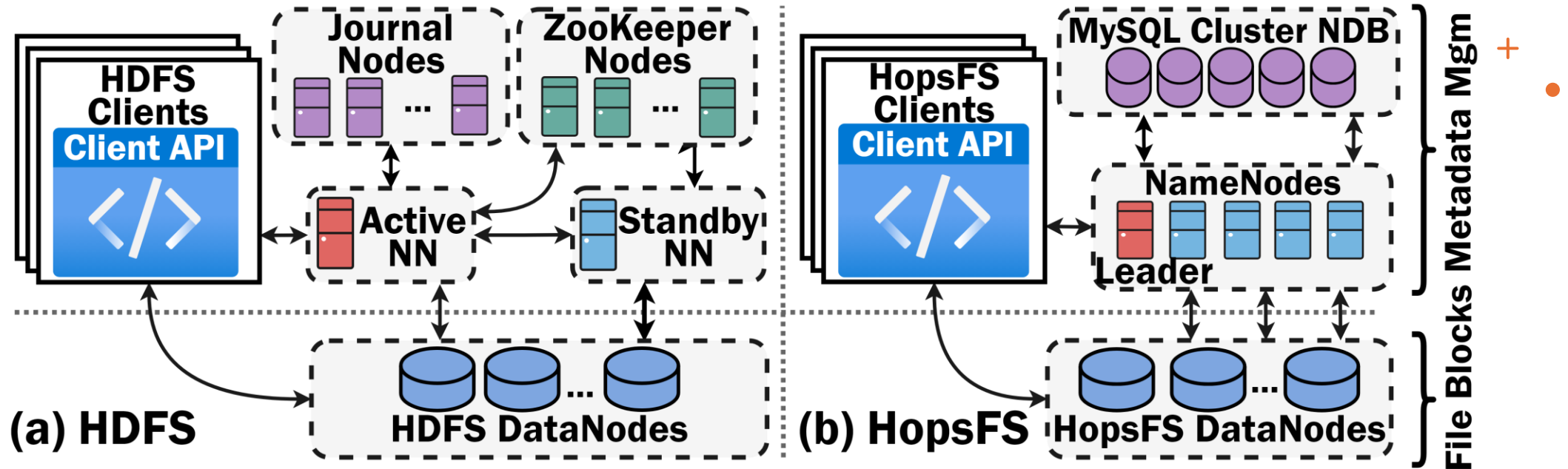
MDS: **key** to the **overall performance** of the DFS.

Scaling the **MDS** is **hard**. Existing scaling techniques:

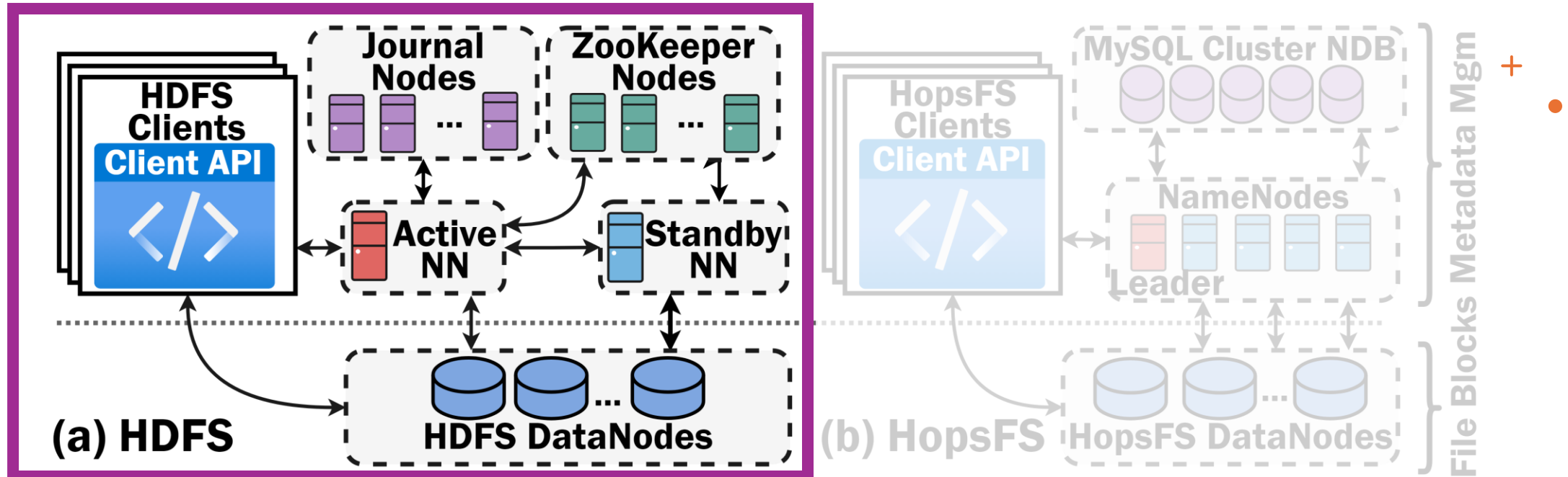
- **Lack elasticity.**
- **Require tedious server management.**
- **May suffer from high costs & low resource utilization.**



Existing Systems:

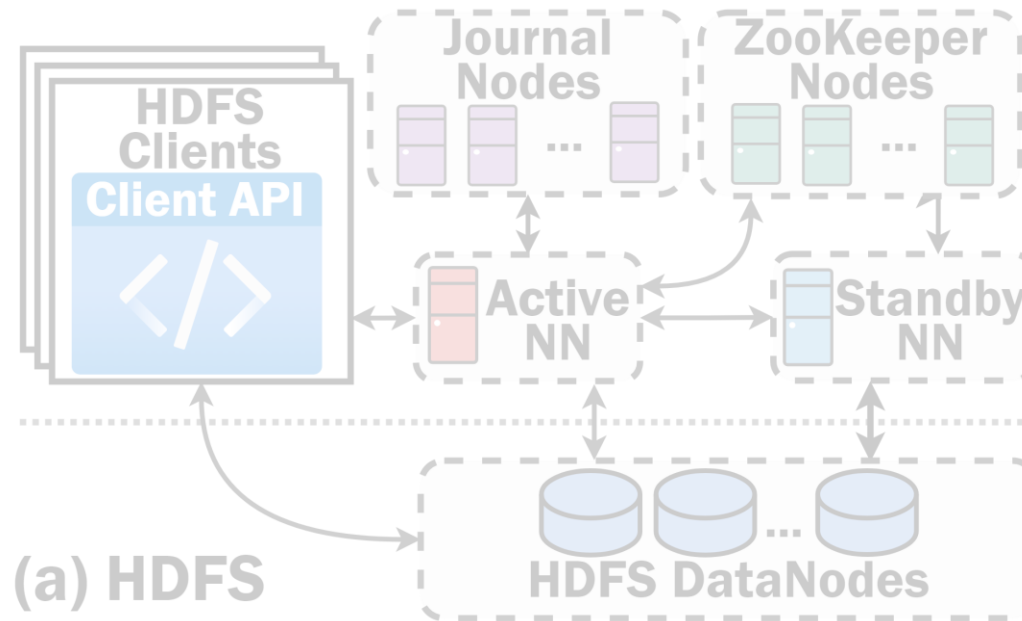


Existing Systems: HDFS

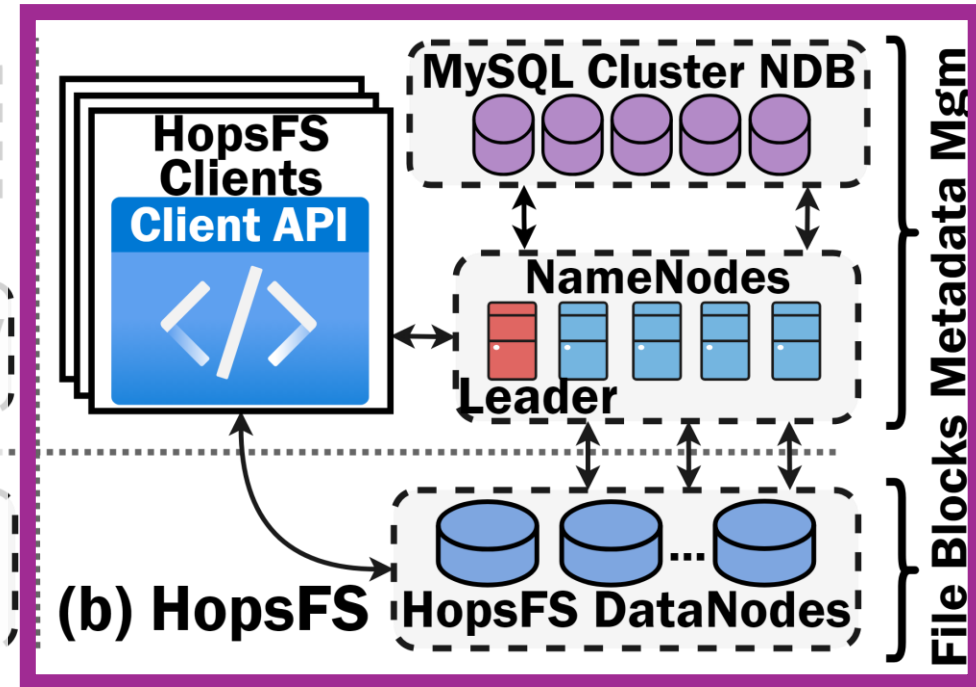


- **Hadoop Distributed File System (HDFS):** open-source implementation of Google File System (GFS).
- **HDFS stores metadata in the Active NameNode (ANN).**

Existing Systems: HopsFS



(a) HDFS



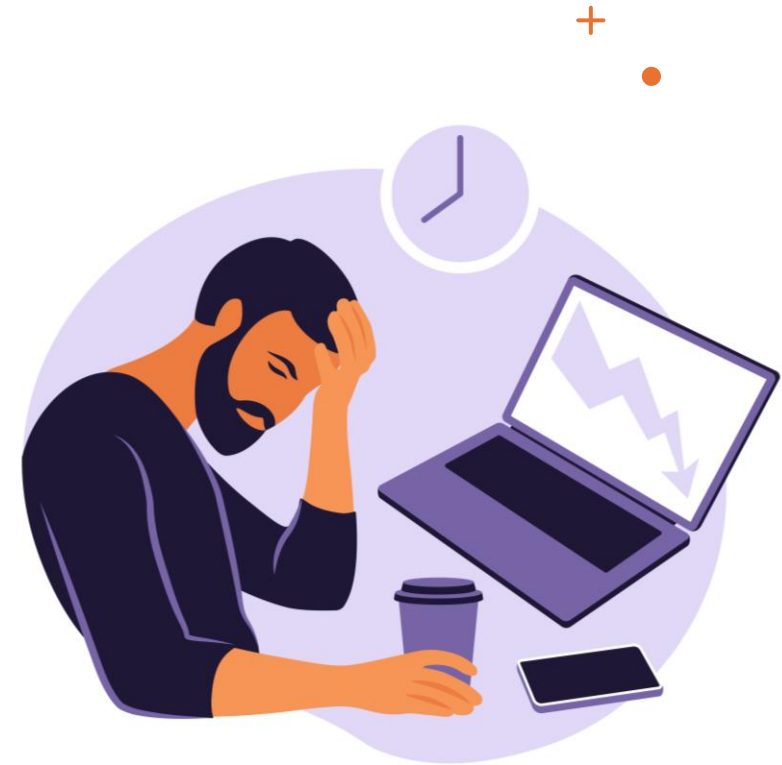
(b) HopsFS

- **HopsFS¹ extends HDFS:**
 - Scaled-out MDS consisting of ≥ 1 stateless NameNodes (NNs).
 - Achieved by **decoupling storage** and **manipulation of metadata**.
- **NNs persist metadata to pluggable storage backend & collectively serve metadata requests.**
 - HopsFS uses MySQL Cluster NDB.

Limitations of Today's Scaled-Out Metadata Services

Focus: HopsFS

1. **Stateless NameNodes** must access persistent metadata store on every single metadata operation.



Stateless NameNodes must access persistent metadata store on every single metadata operation.

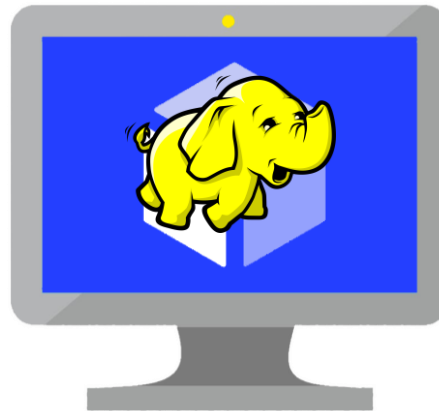
+

●

Stateless NameNodes must access persistent metadata store on every single metadata operation.



HopsFS Client

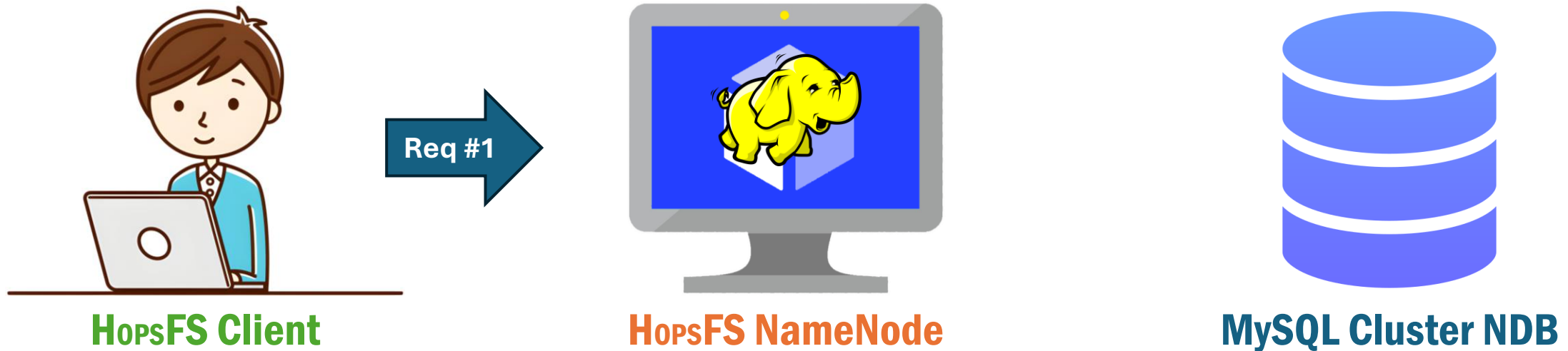


HopsFS NameNode

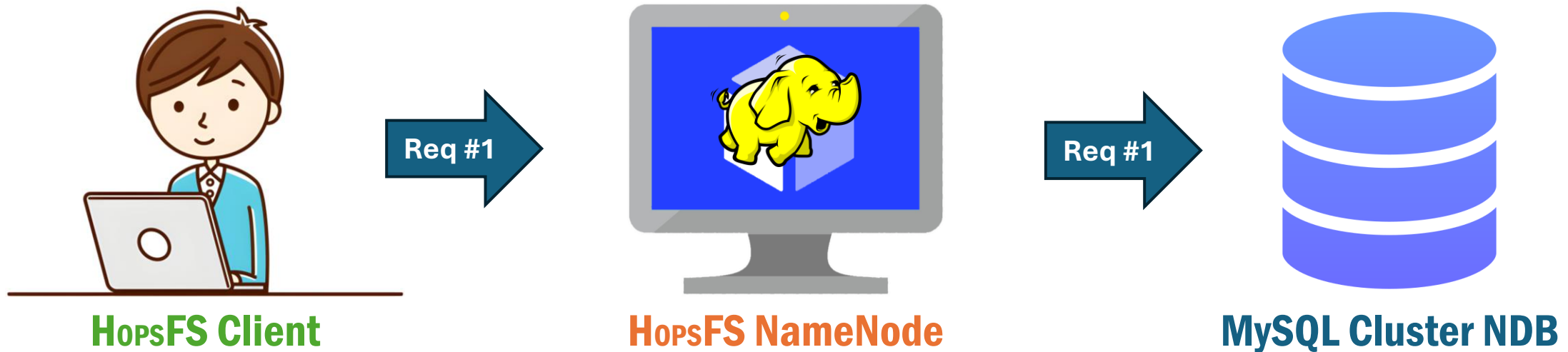


MySQL Cluster NDB

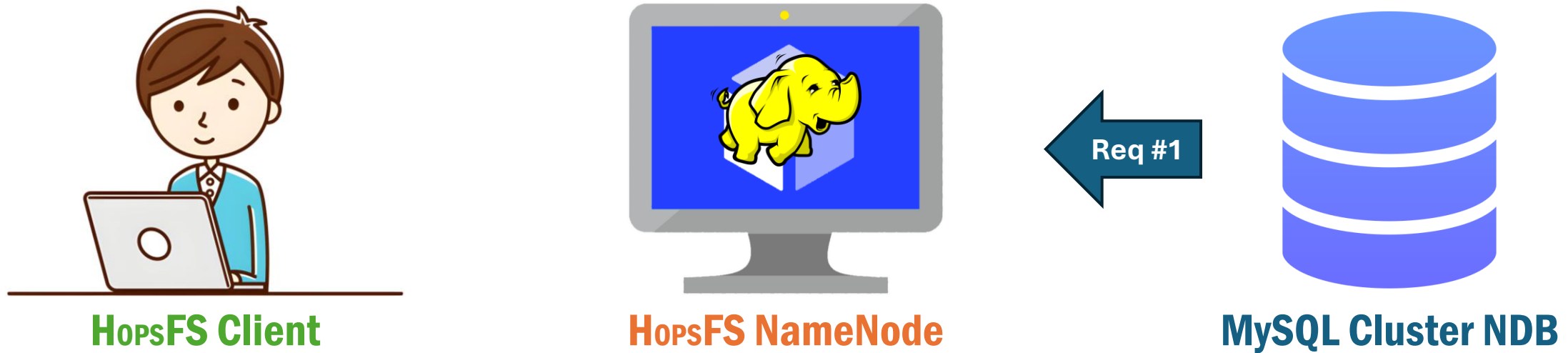
Stateless NameNodes must access persistent metadata store on every single metadata operation.



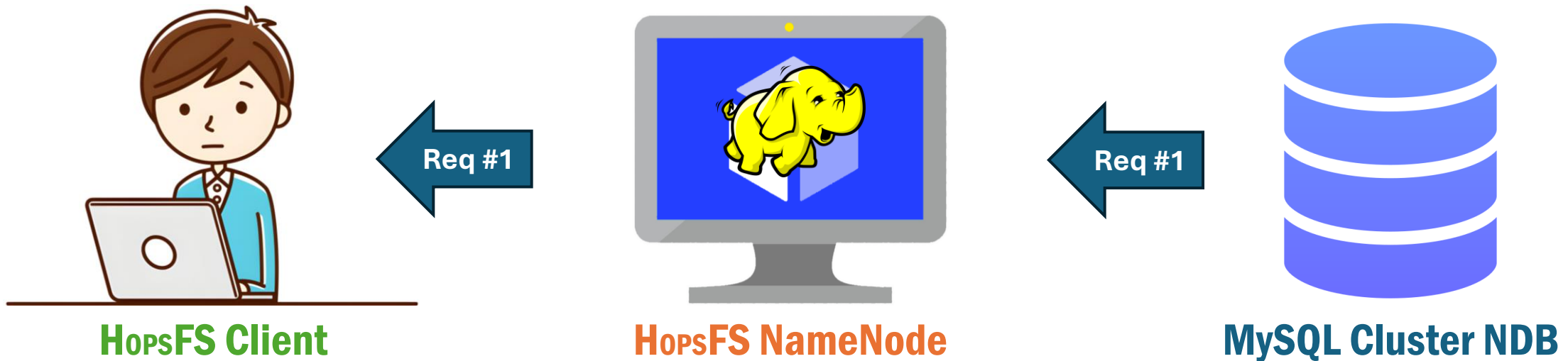
Stateless NameNodes must access persistent metadata store on every single metadata operation.



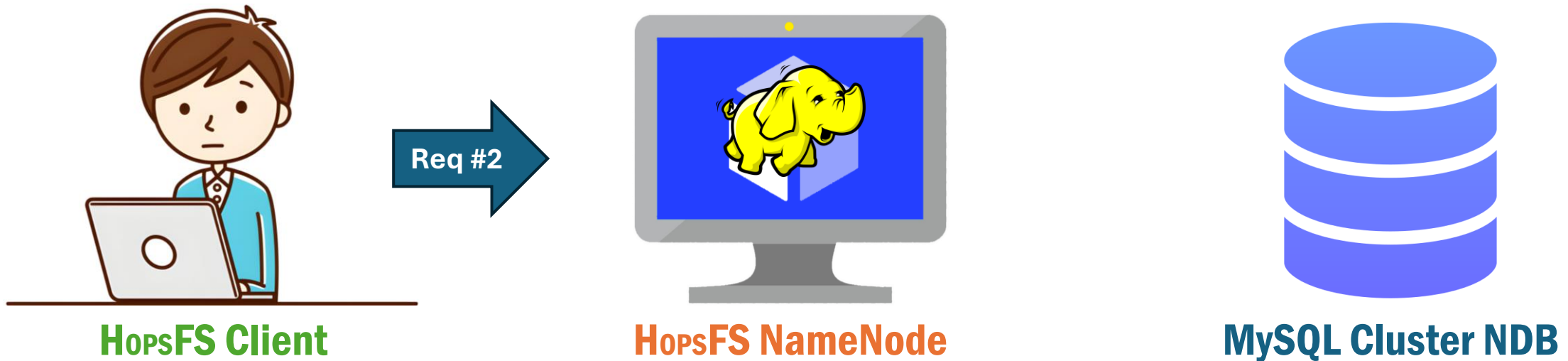
Stateless NameNodes must access persistent metadata store on every single metadata operation.



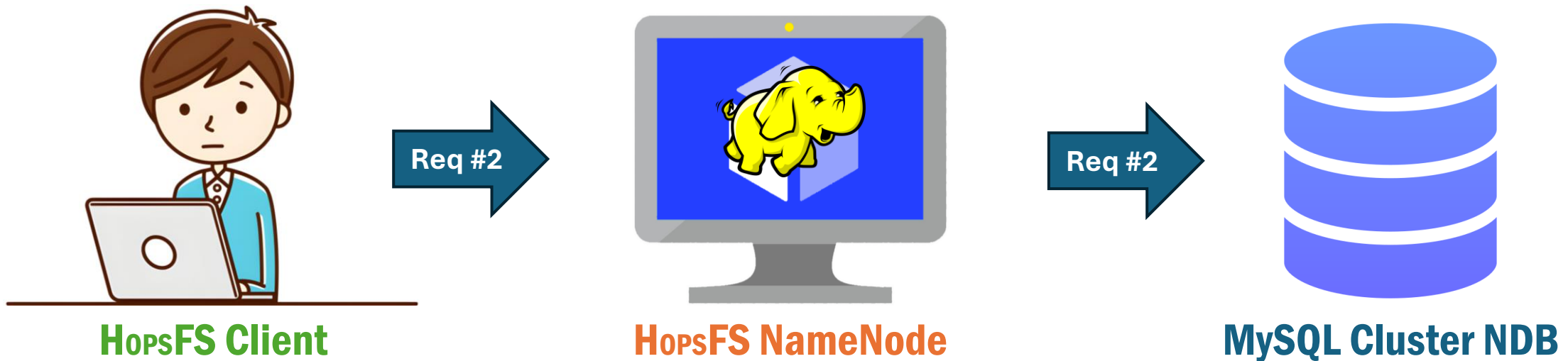
Stateless NameNodes must access persistent metadata store on every single metadata operation.



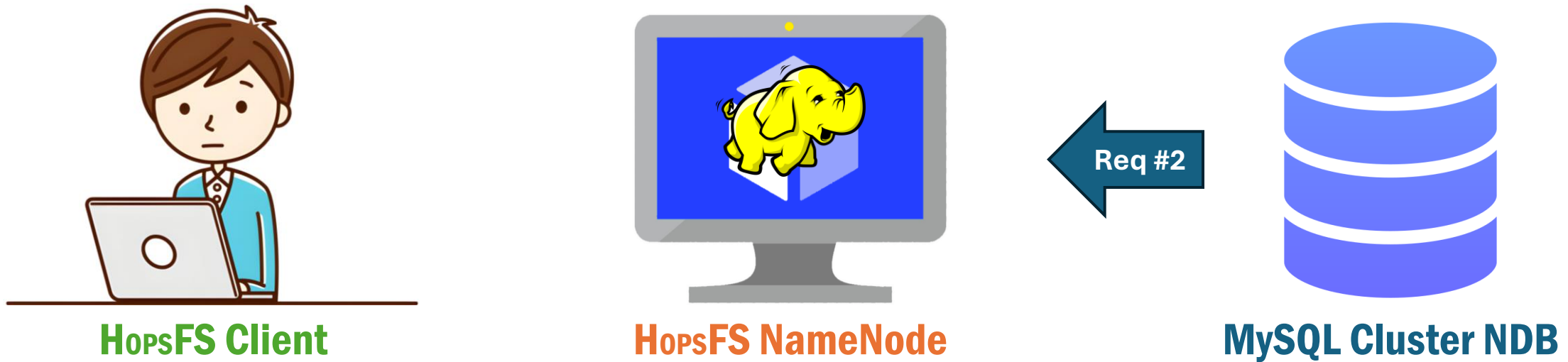
Stateless NameNodes must access persistent metadata store on every single metadata operation.



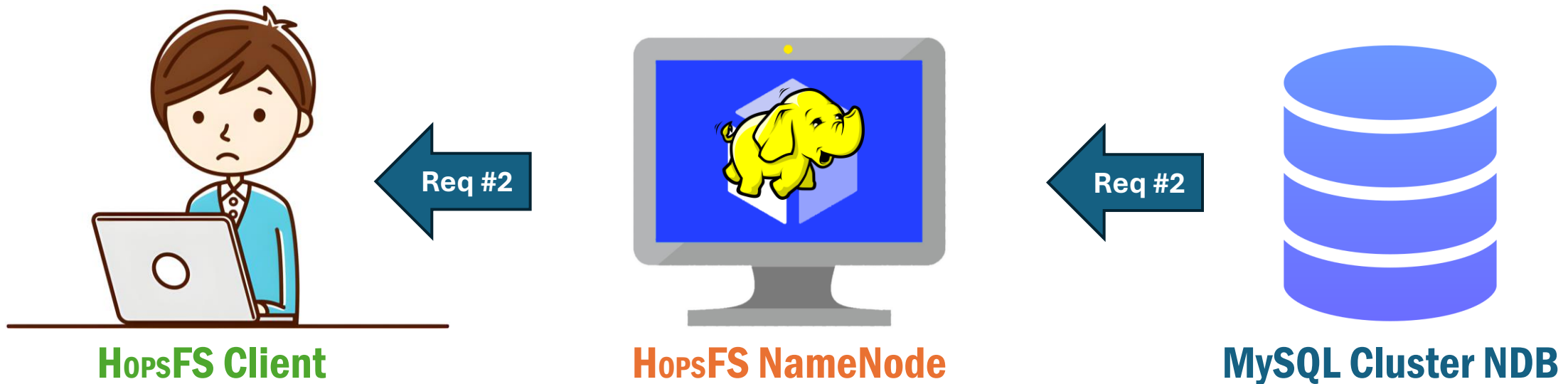
Stateless NameNodes must access persistent metadata store on every single metadata operation.



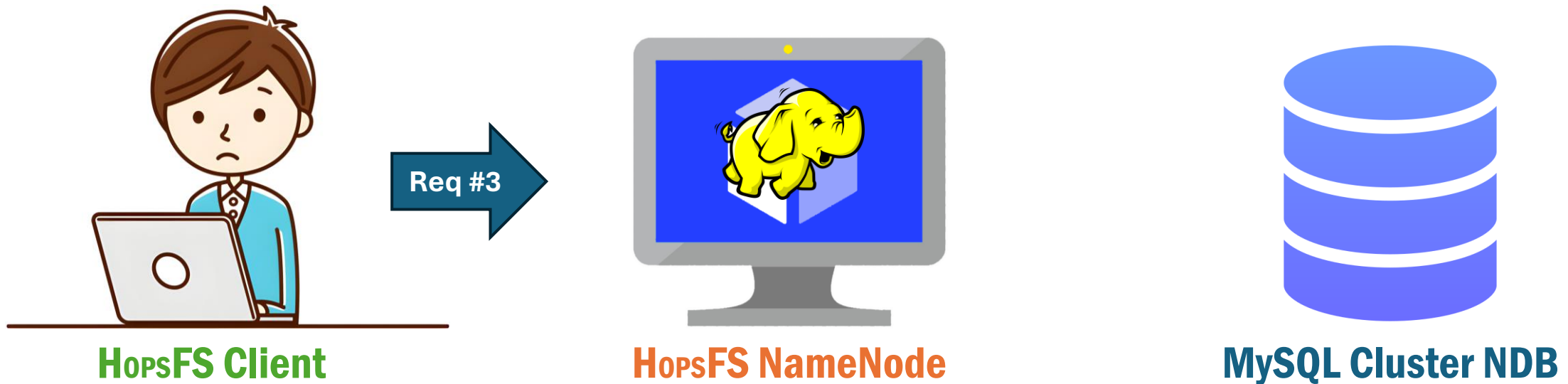
Stateless NameNodes must access persistent metadata store on every single metadata operation.



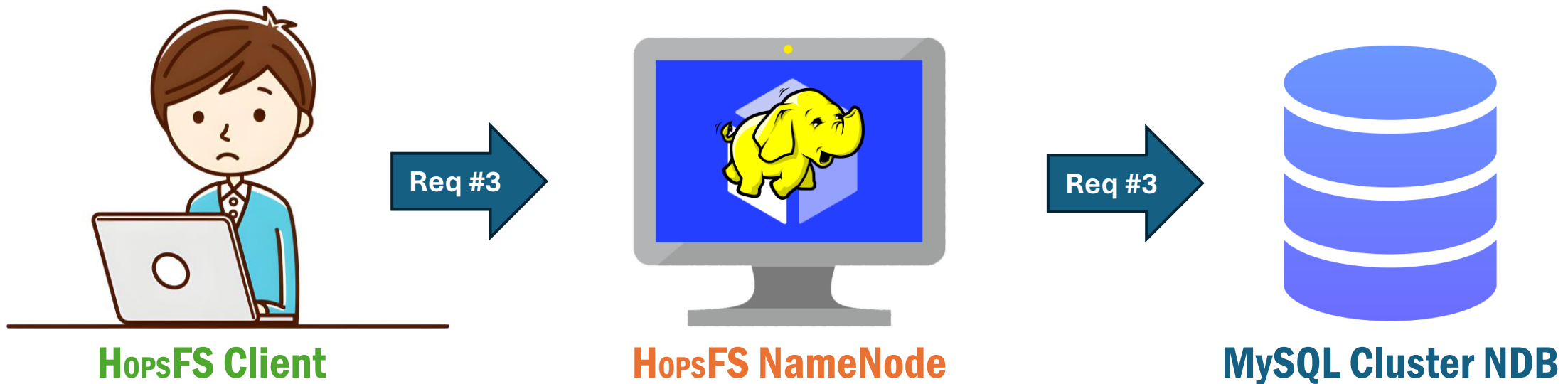
Stateless NameNodes must access persistent metadata store on every single metadata operation.



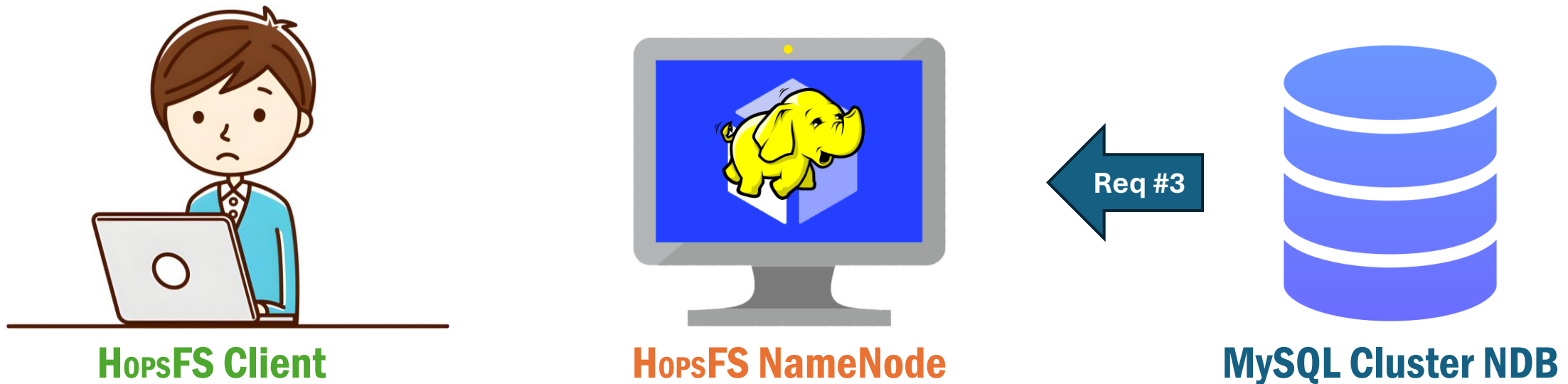
Stateless NameNodes must access persistent metadata store on every single metadata operation.



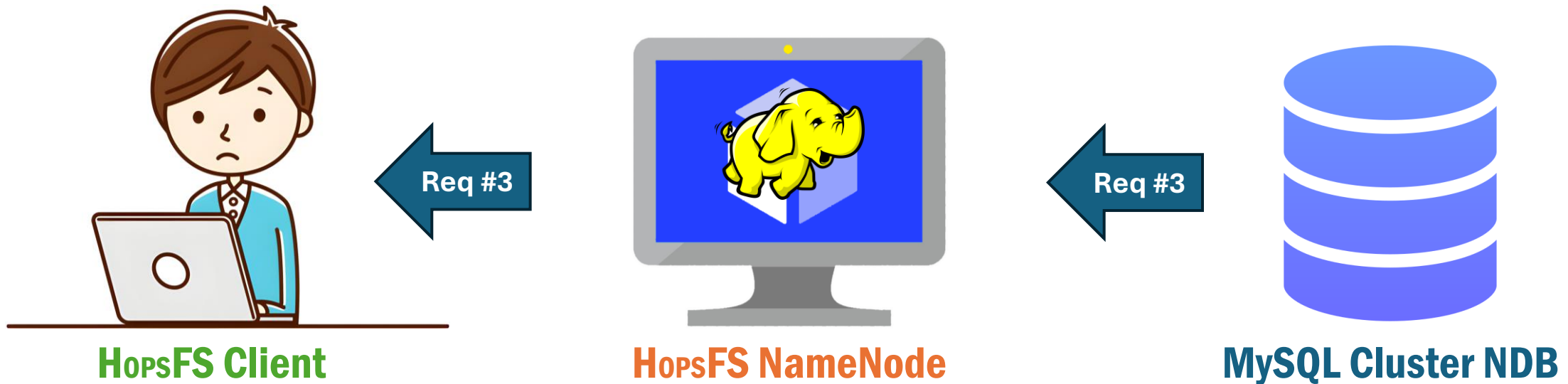
Stateless NameNodes must access persistent metadata store on every single metadata operation.



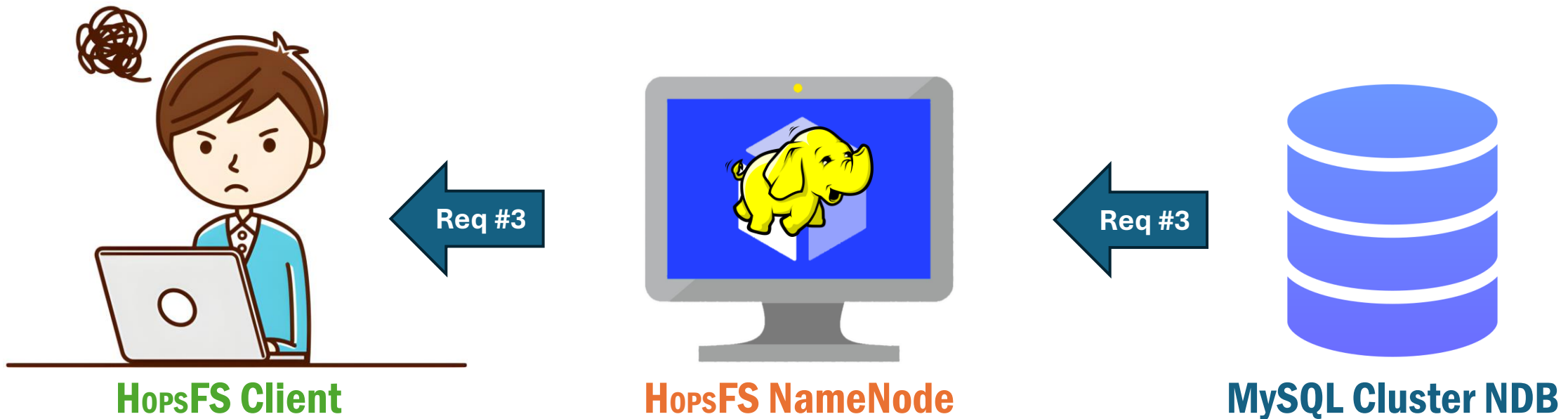
Stateless NameNodes must access persistent metadata store on every single metadata operation.



Stateless NameNodes must access persistent metadata store on every single metadata operation.



Stateless NameNodes must access persistent metadata store on every single metadata operation.

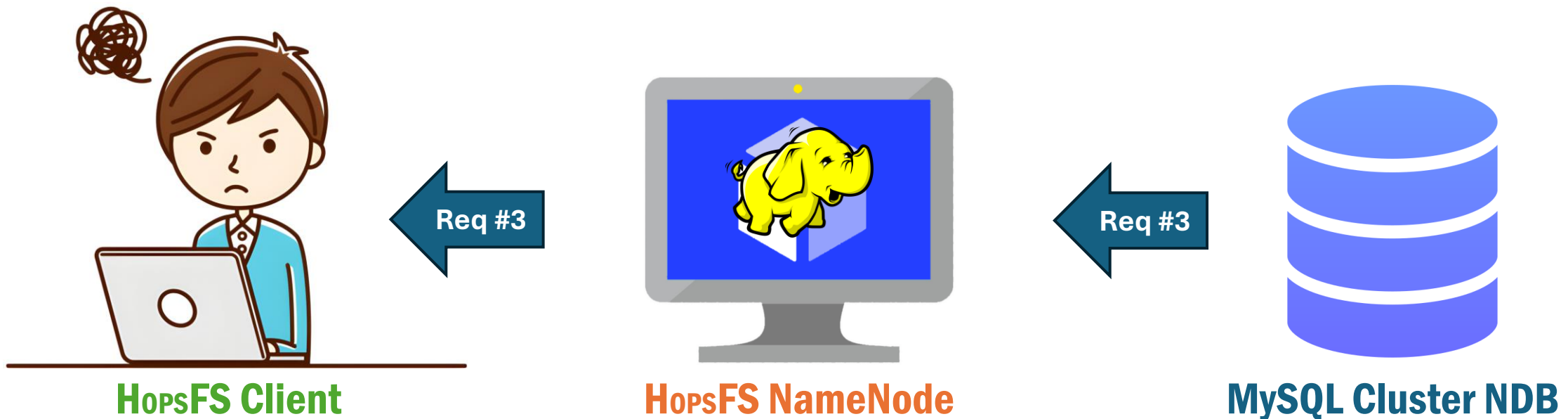


Stateless NameNodes must access persistent metadata store on every single metadata operation.



RESULT: Performance **capped** by capacity of the backend NDB cluster.

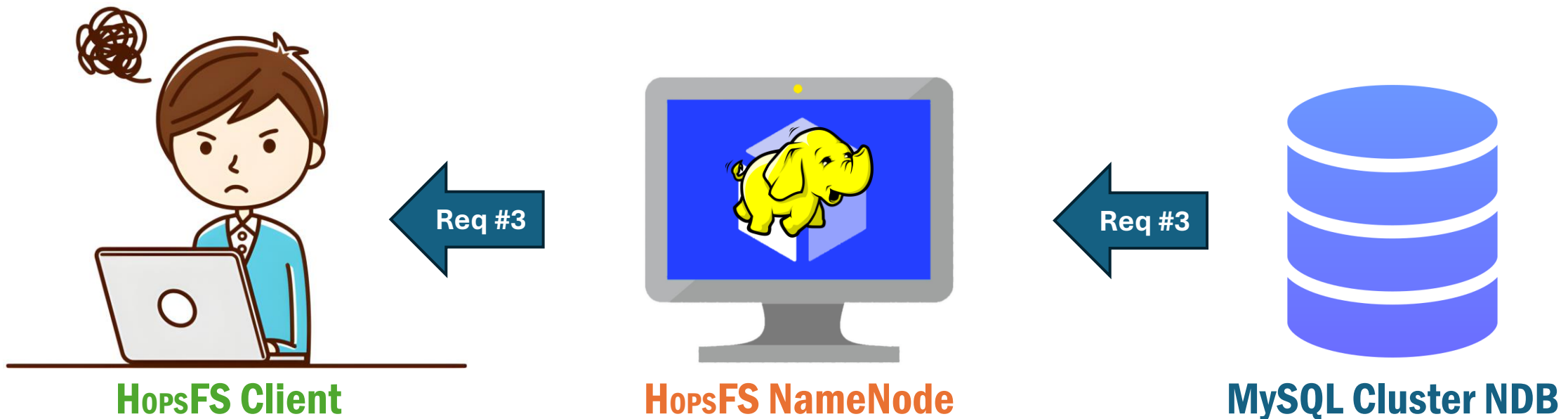
Stateless NameNodes must access persistent metadata store on every single metadata operation.



RESULT: Performance **capped** by capacity of the backend NDB cluster.

Compute (**NameNodes**) and **storage** (**NDB**) are physically decoupled but *logically-bundled*.

Stateless NameNodes must access persistent metadata store on every single metadata operation.



RESULT: Performance **capped** by capacity of the backend NDB cluster.

Compute (NameNodes) and **storage** (NDB) are physically decoupled but *logically-bundled*.

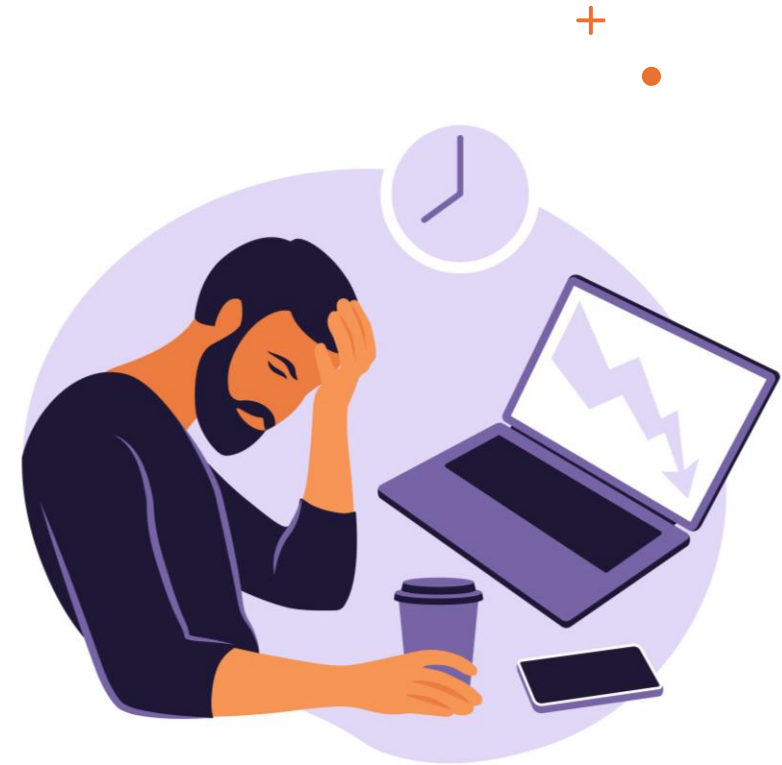
Performance can **rapidly degrade** if either layer becomes a bottleneck.

Limitations of Today's Scaled-Out Metadata Services

Focus: HopsFS

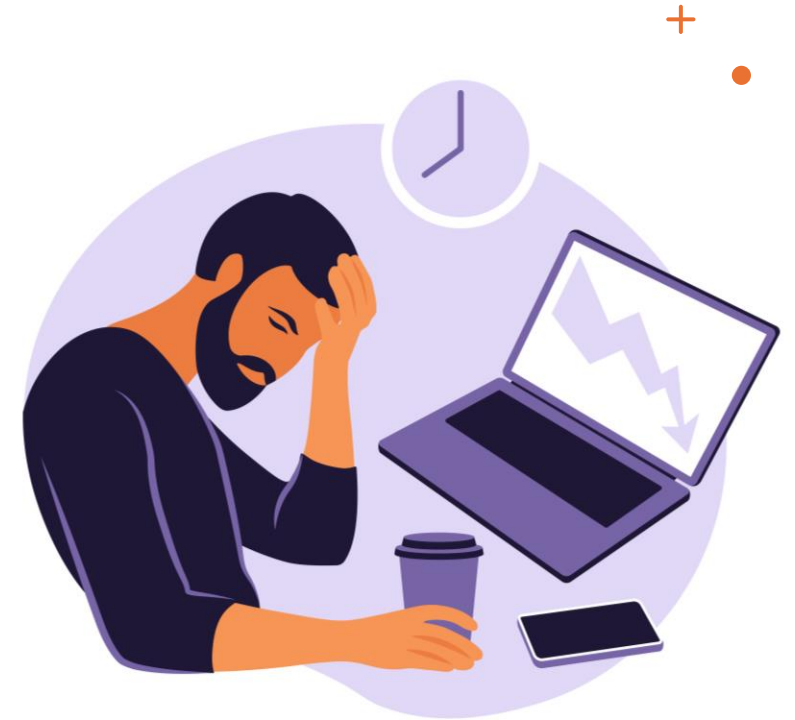
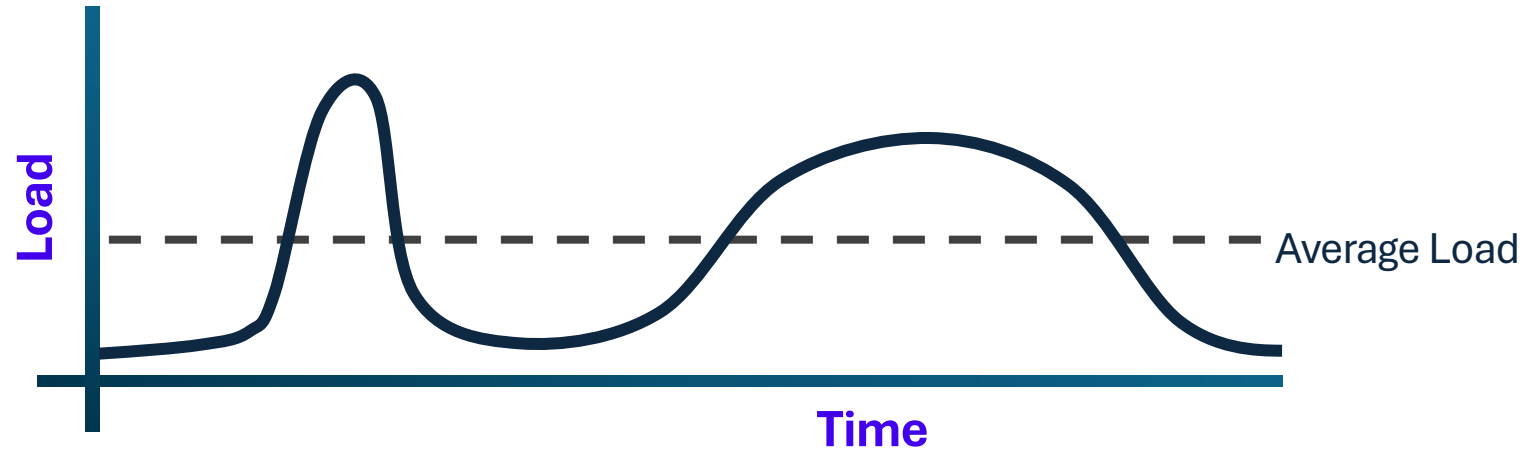
1. **Stateless NameNodes** must access persistent metadata store on every single metadata operation.
2. HopsFS (and other scaled-out MDSeS) **lack elasticity**.

Admin must **statically** configure resources.

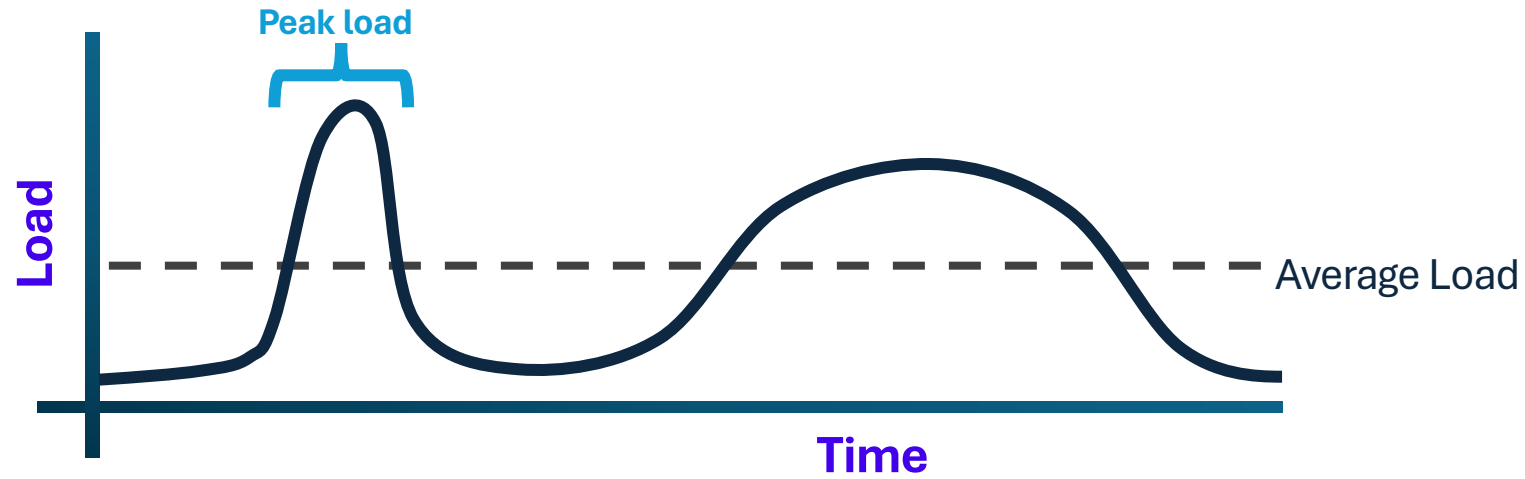


Resources must be *statically configured*.

λFS BACKGROUND

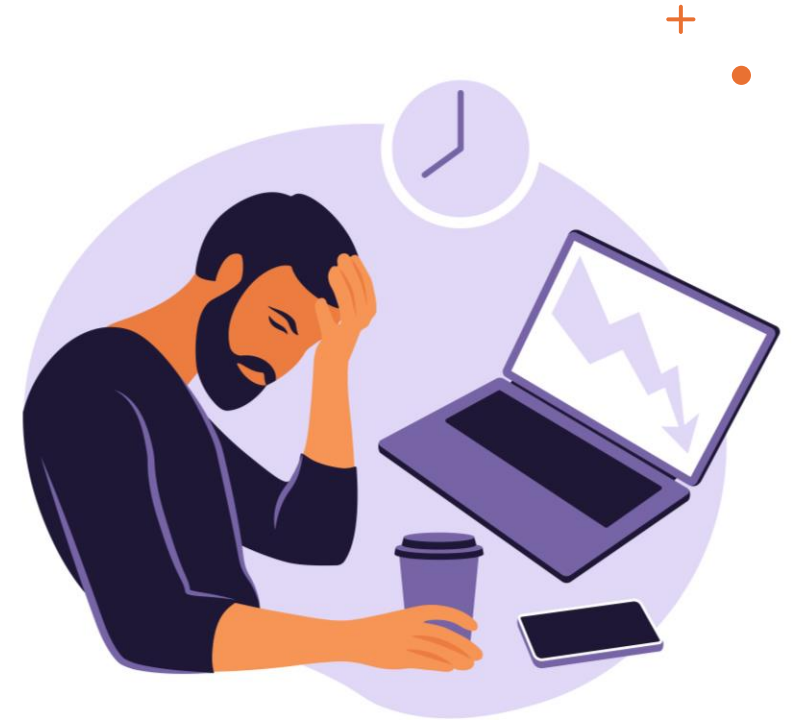


Resources must be *statically configured*.



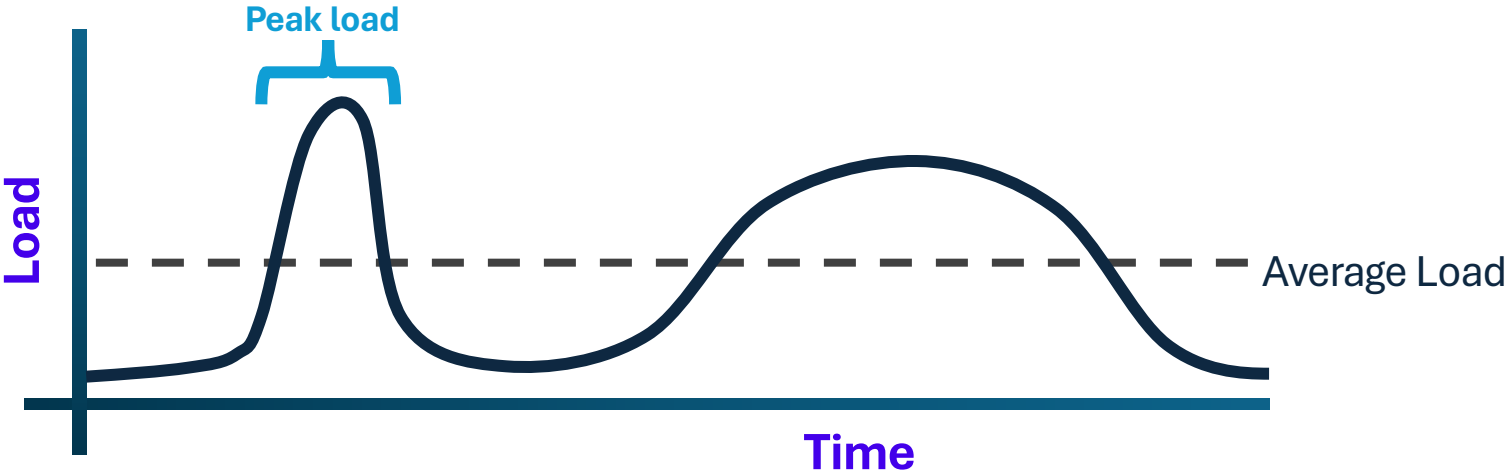
Provisioned for peak load

λFS BACKGROUND

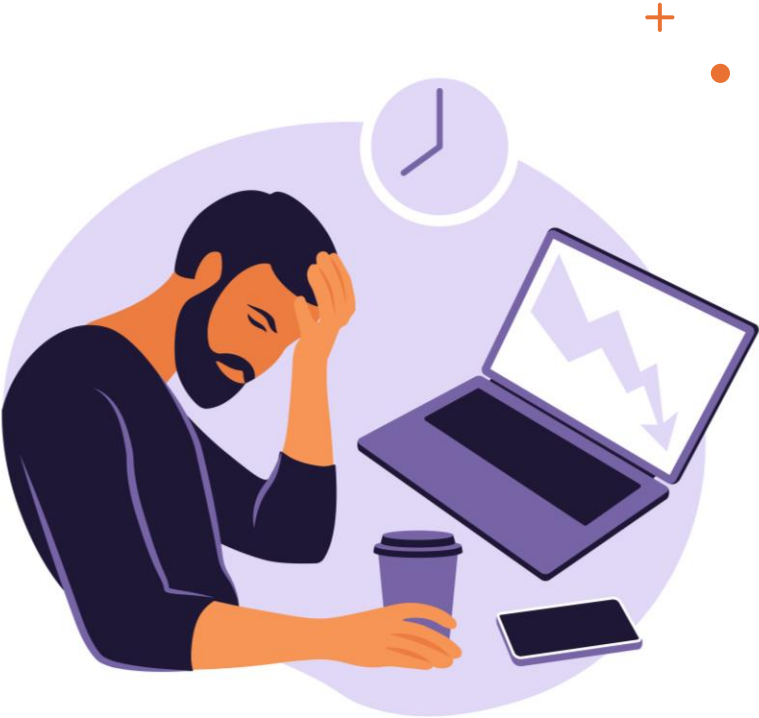
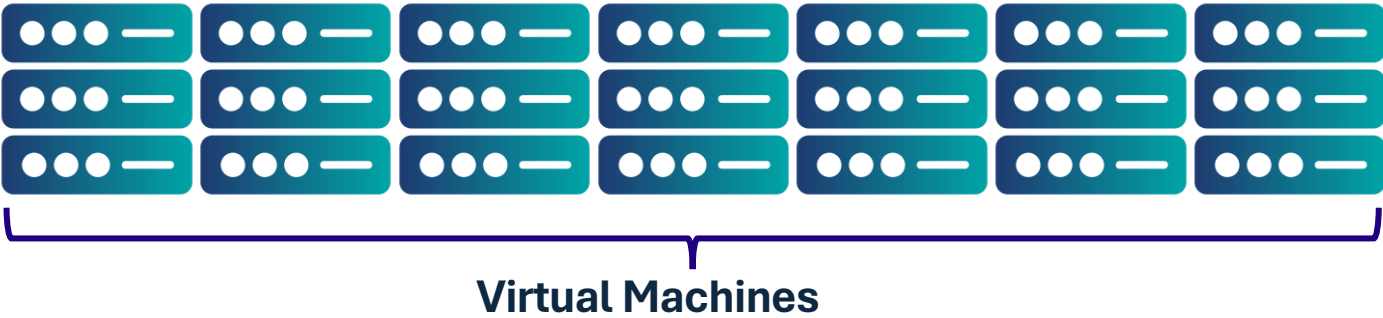


Resources must be *statically configured*.

λFS BACKGROUND

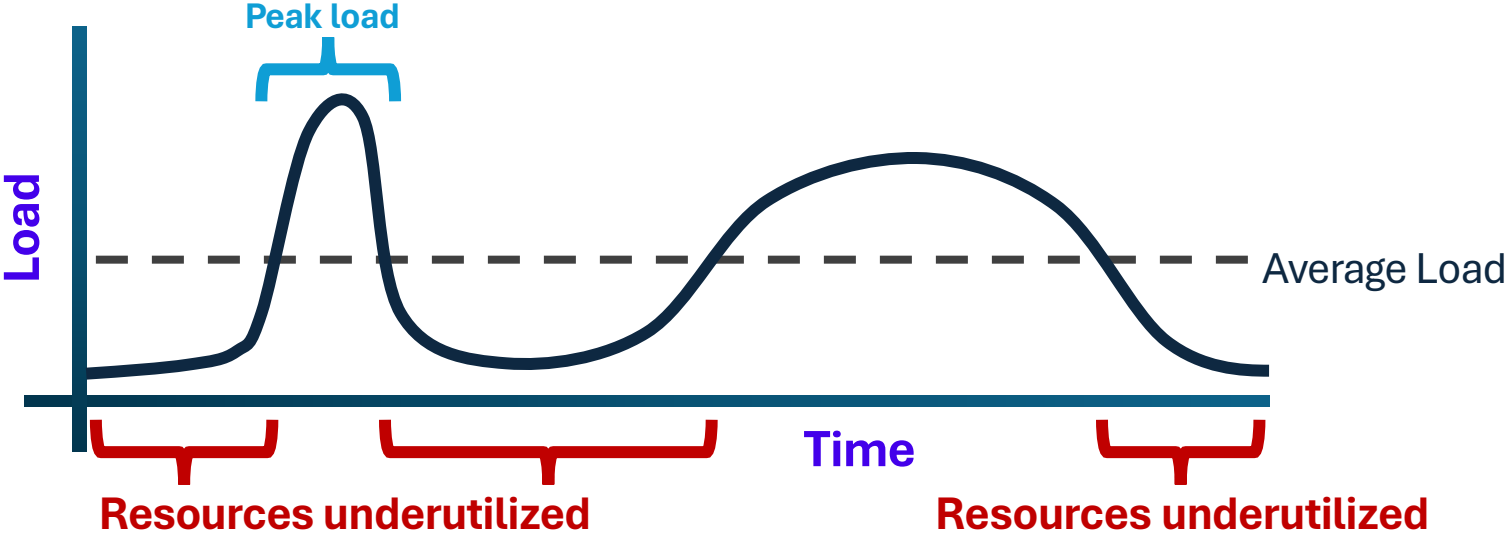


Provisioned for peak load

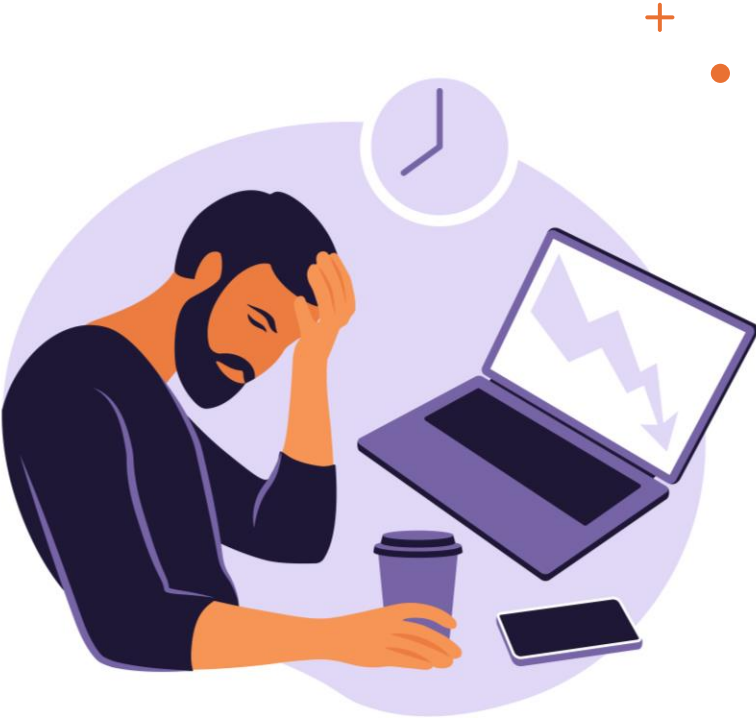
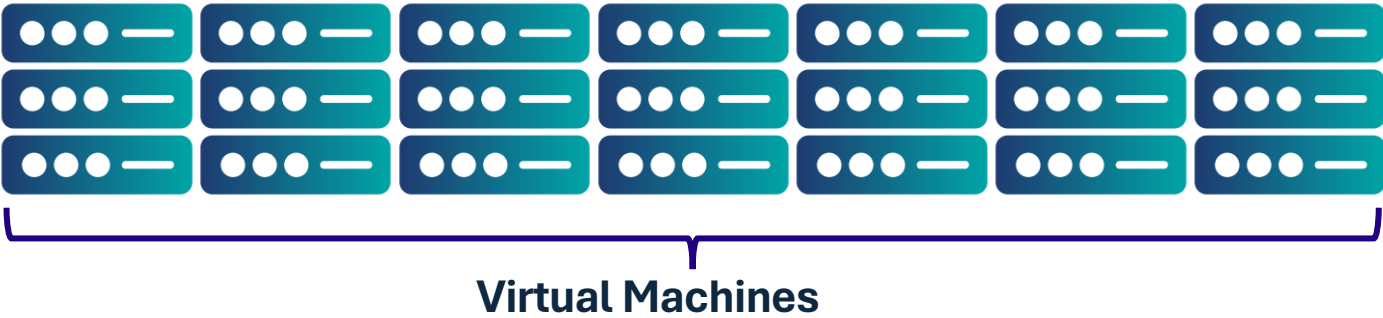


Resources must be *statically configured*.

λFS BACKGROUND

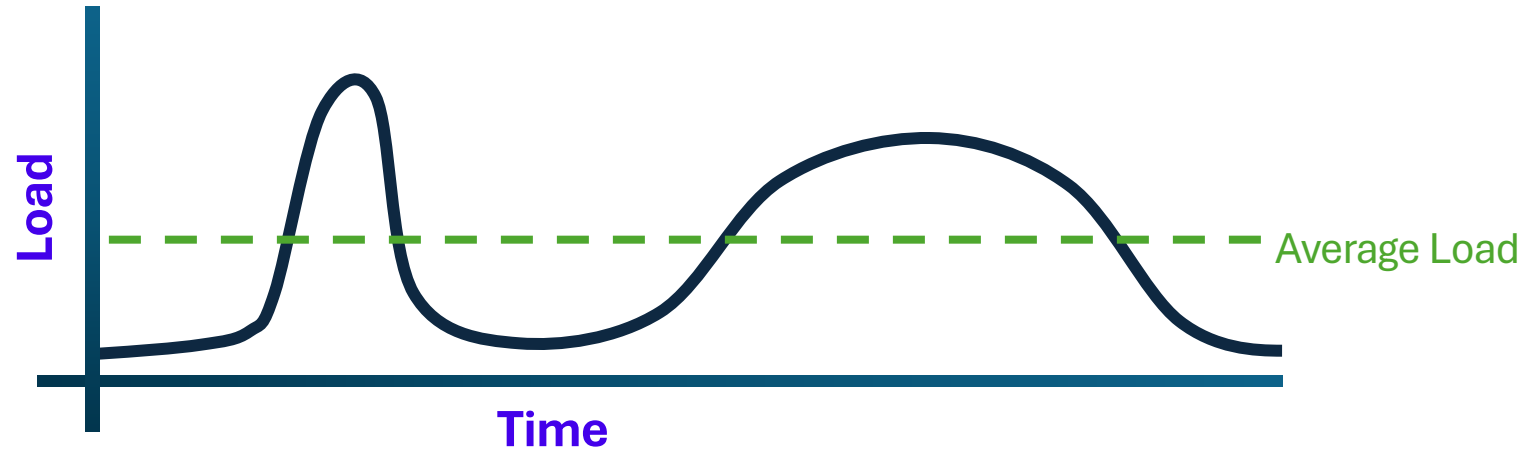


Provisioned for peak load

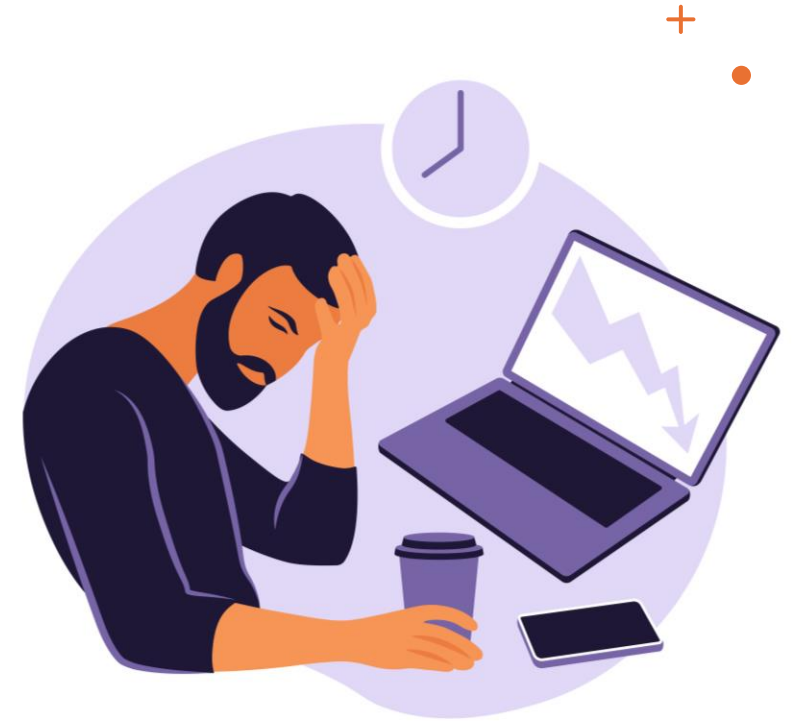


Resources must be *statically configured*.

λFS BACKGROUND

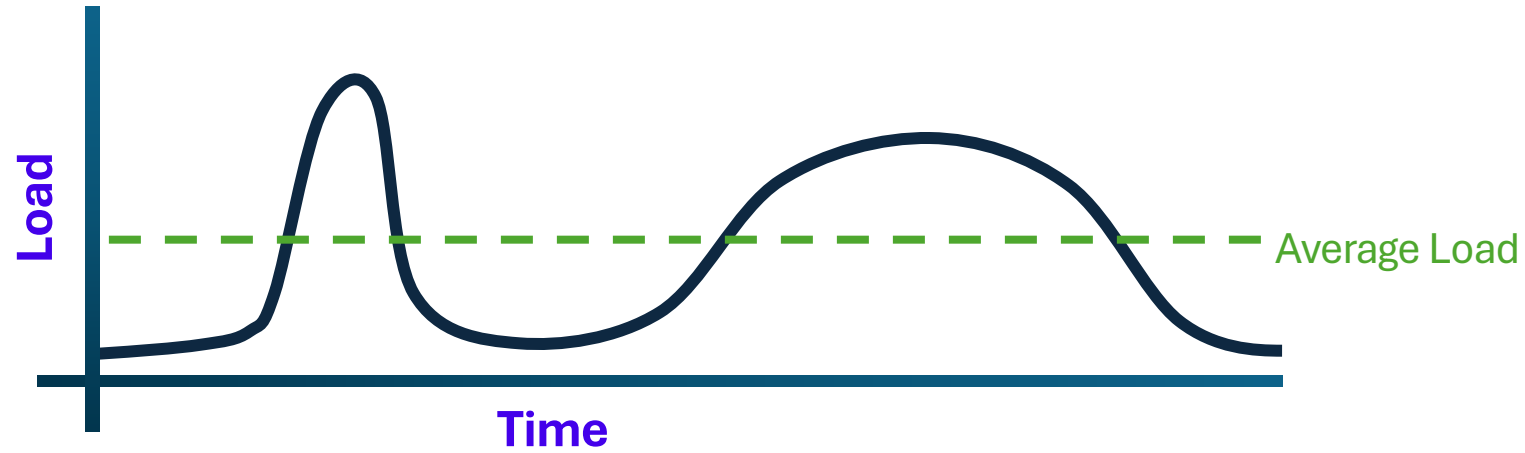


Provisioned for **average load**

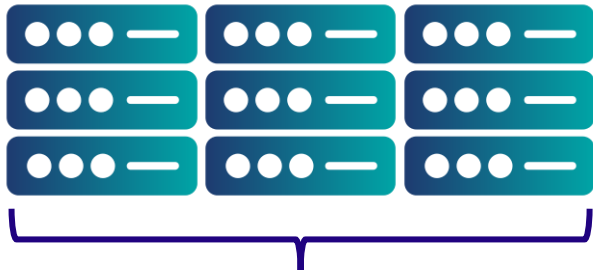


Resources must be *statically configured*.

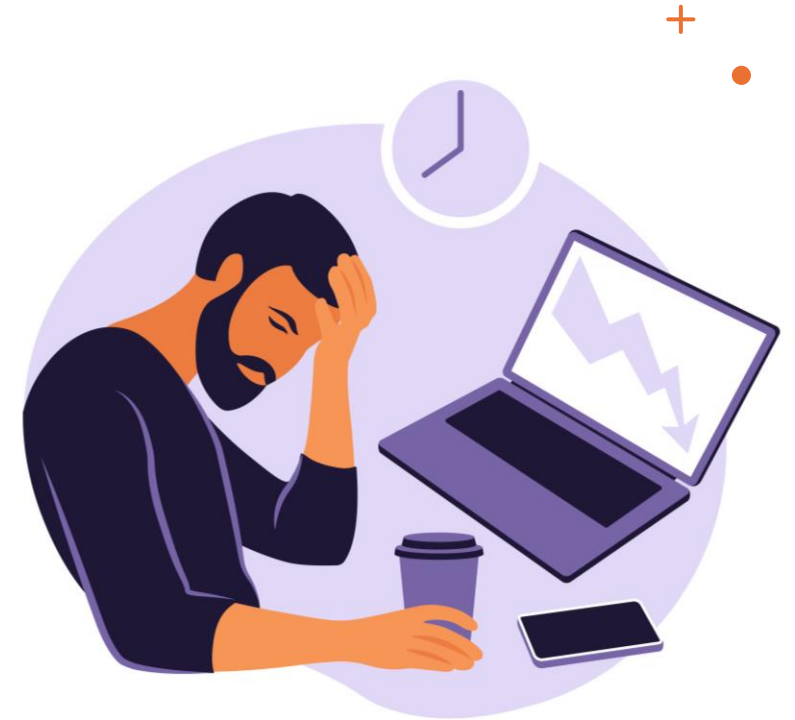
λFS BACKGROUND



Provisioned for **average load**

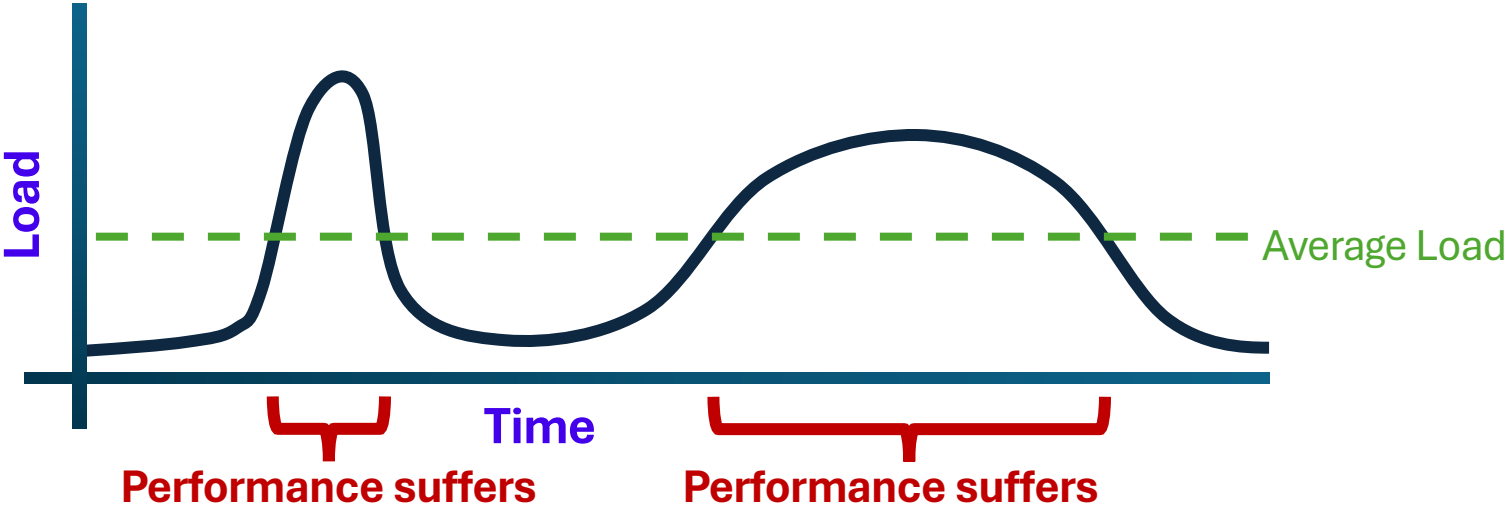


Virtual Machines



Resources must be *statically configured*.

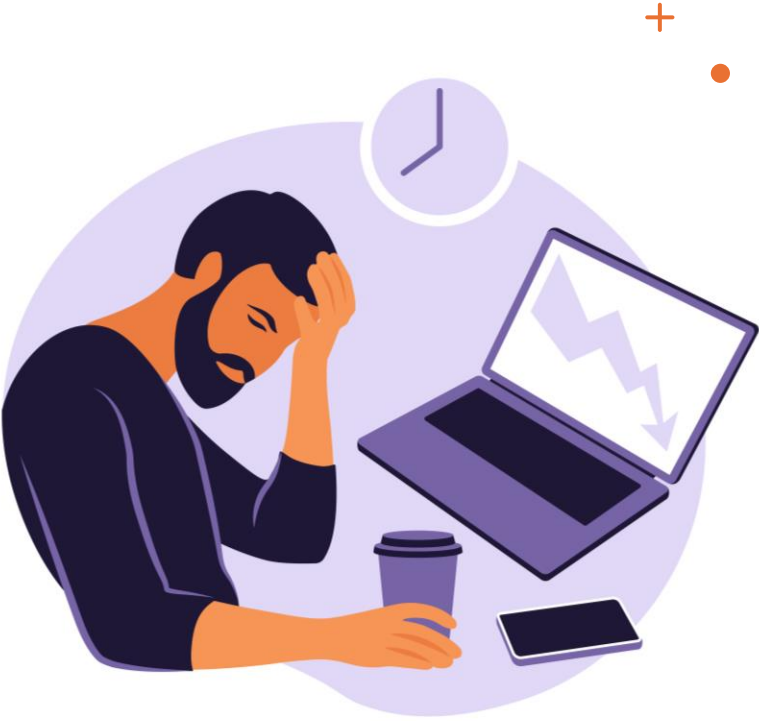
λFS BACKGROUND



Provisioned for **average load**



Virtual Machines



Can we use innovative FaaS-based design mechanisms to build a high-performance, cost-effective, elastic, and resource-efficient MDS?



Yes!

Serverless functions are an **appealing environment** in which to host the metadata service.



**FaaS-based elastic
auto-scaling**



**Pay-per-use
pricing model**



**Automatic resource
provisioning & management**

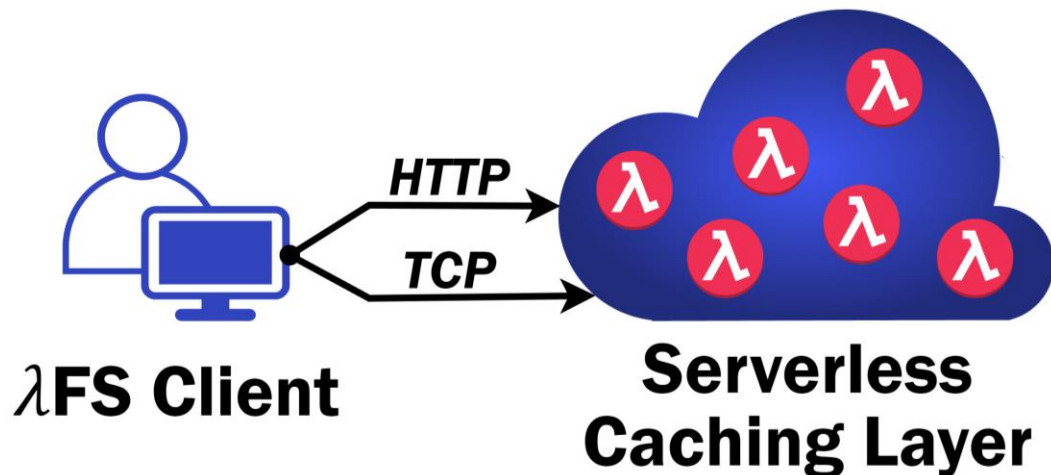
Introducing:



λFS: Lambda File System

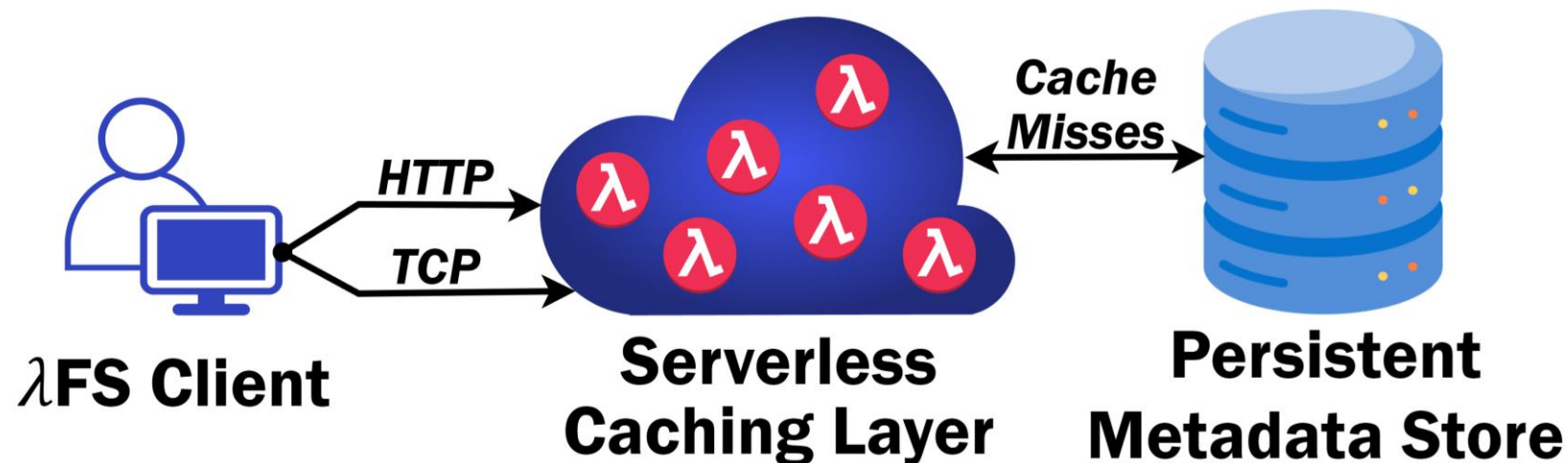
λFS Overview

- ❖ Elastic, scalable, & high-performance MDS for large-scale DFSes⁺ •
- ❖ Hybrid TCP-HTTP RPC mechanism



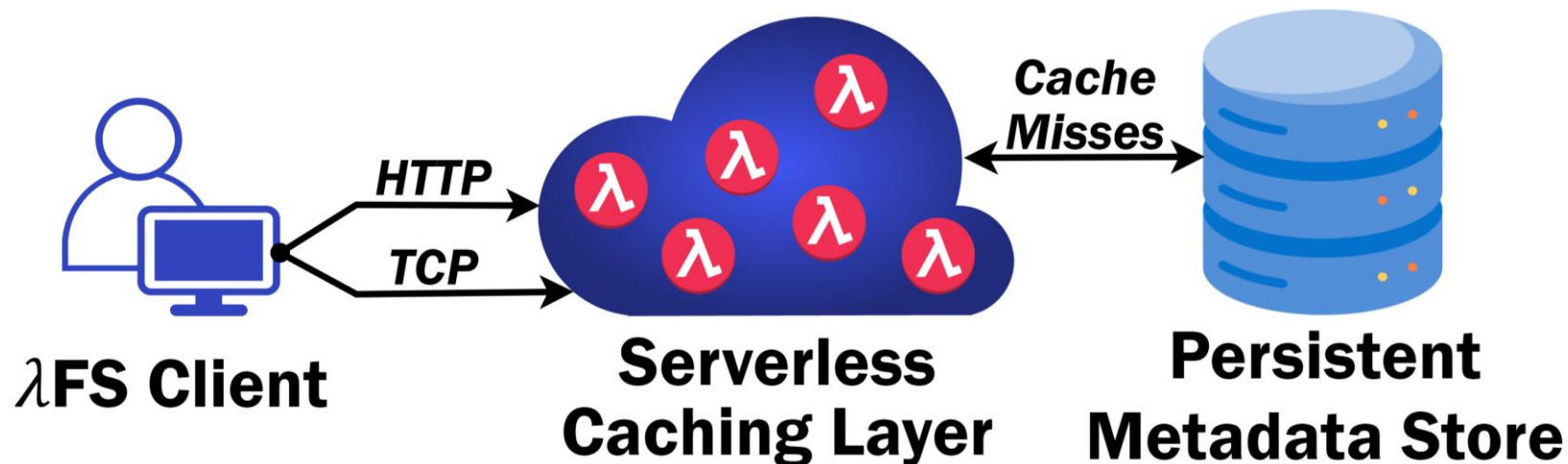
λFS Overview

- ❖ **Elastic, scalable, & high-performance** MDS for large-scale DFSes⁺ •
- ❖ Hybrid **TCP-HTTP RPC** mechanism
- ❖ **Metadata storage** mechanism

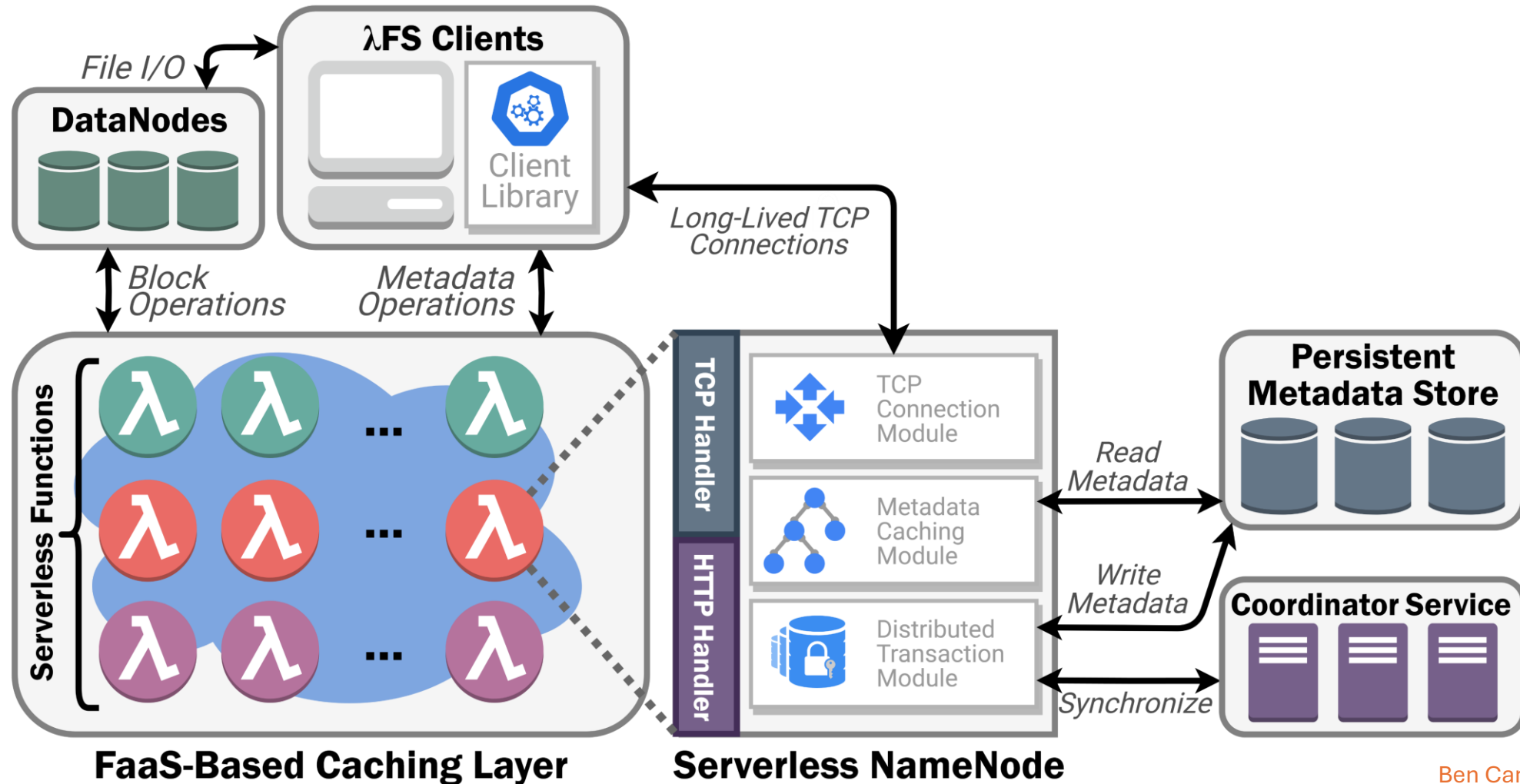


λFS Overview

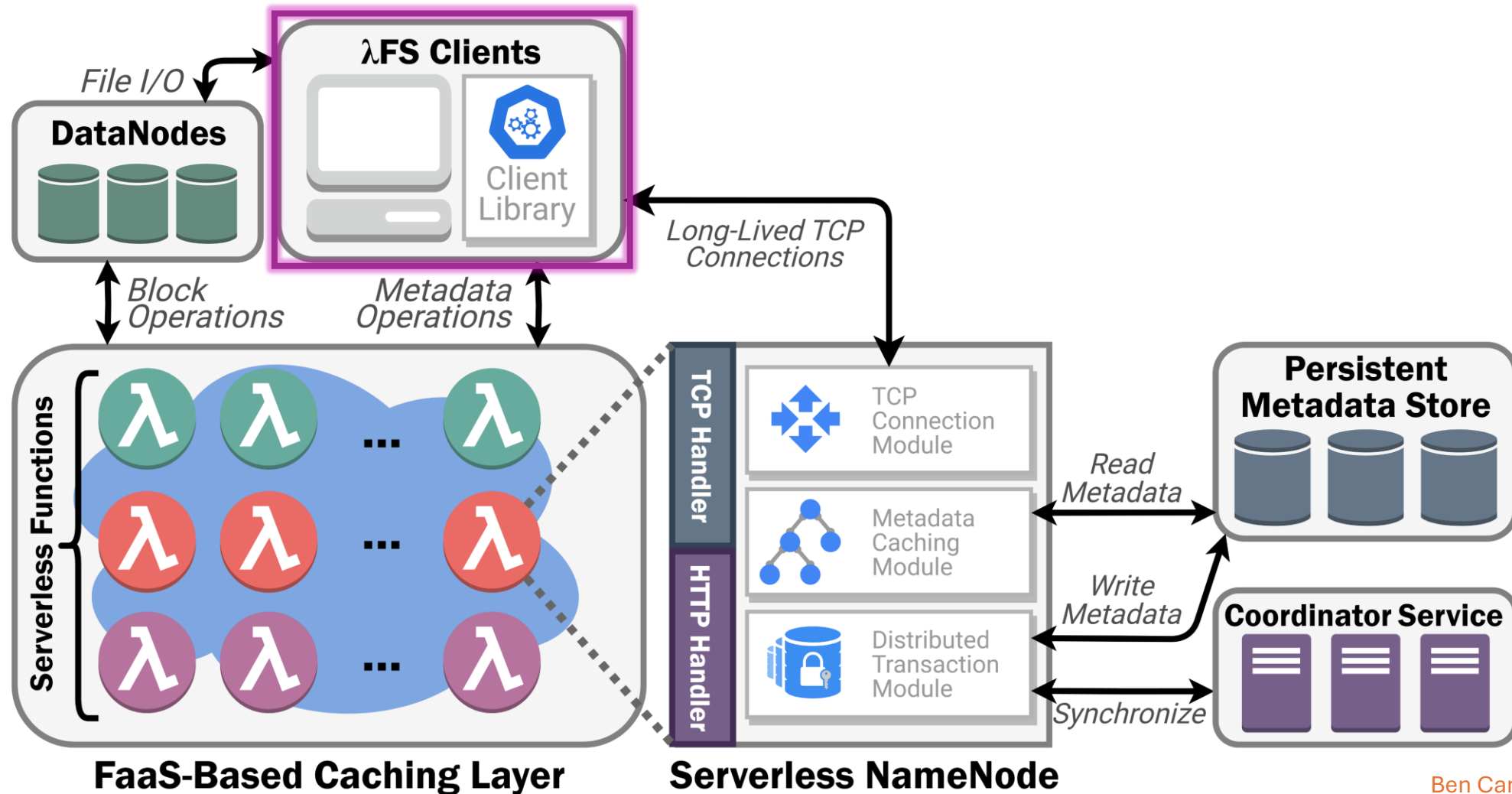
- ❖ **Elastic, scalable, & high-performance** MDS for large-scale DFSes⁺•
- ❖ Hybrid **TCP-HTTP RPC** mechanism
- ❖ **Metadata storage** mechanism
- ❖ Agile **auto-scaling** policy



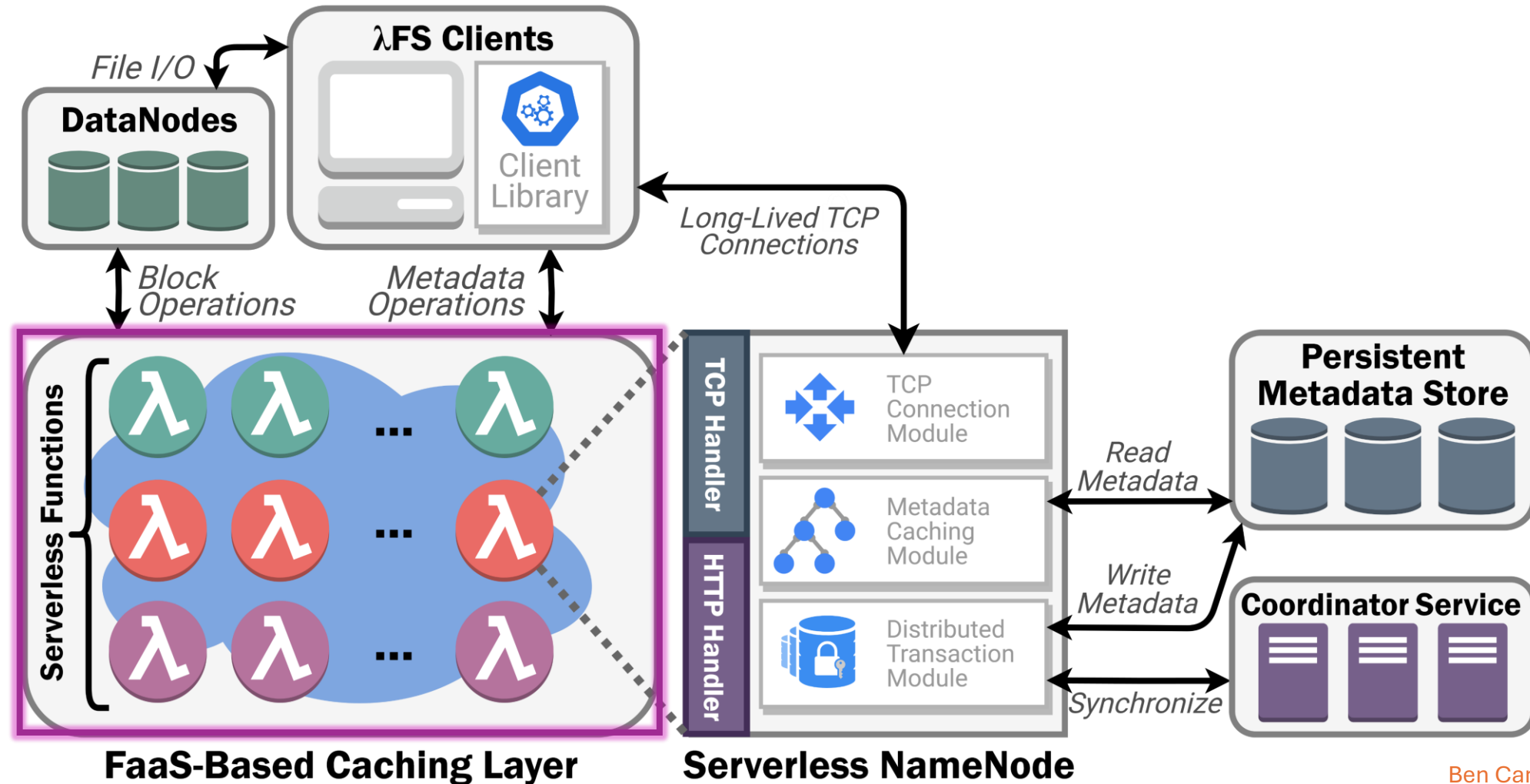
λFS: **Elastic**, **scalable**, and **high-performance** MDS for large-scale DFSes.



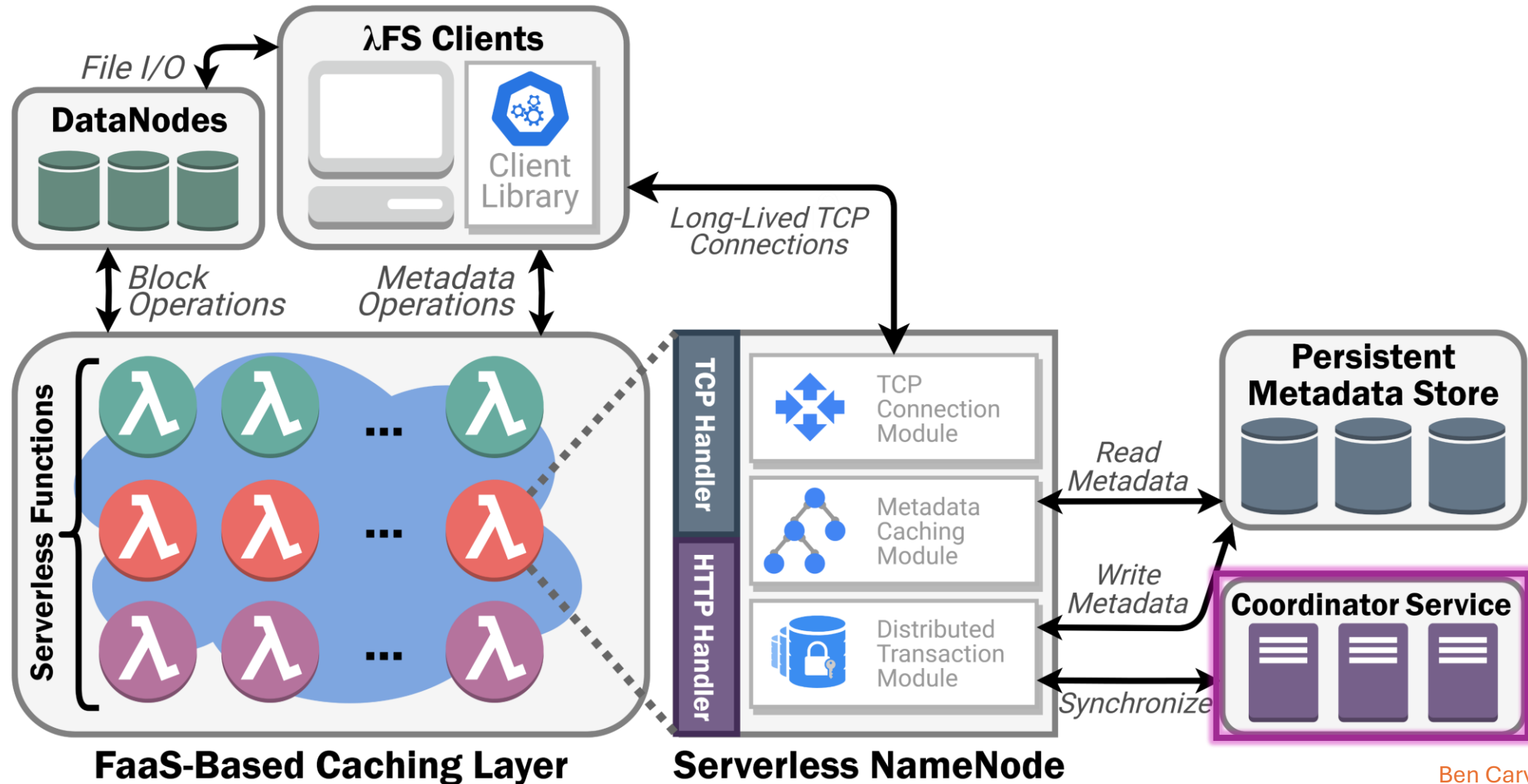
λFS: **Elastic**, **scalable**, and **high-performance** MDS for large-scale DFSes.



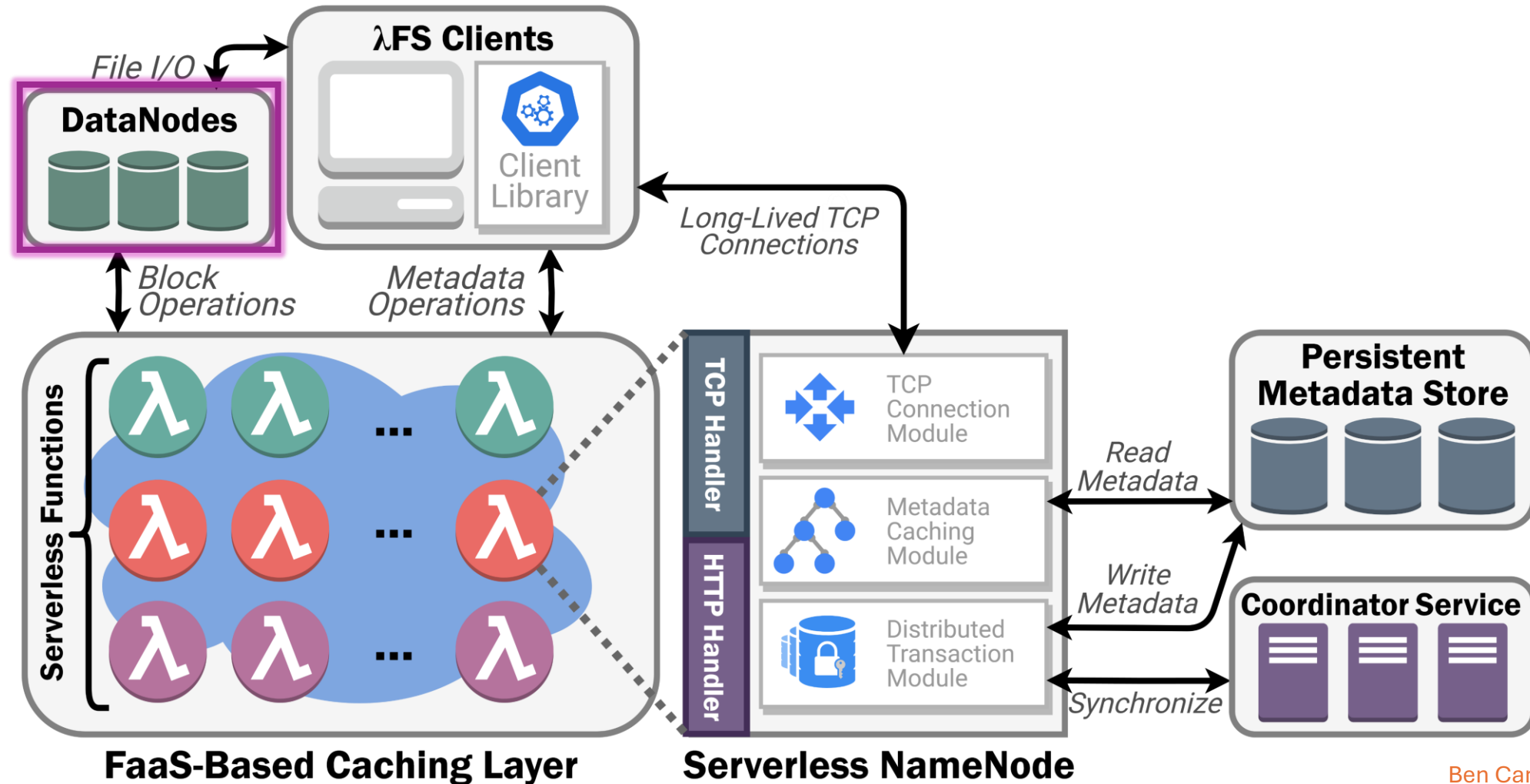
λFS: **Elastic**, **scalable**, and **high-performance** MDS for large-scale DFSes.



λFS: **Elastic**, **scalable**, and **high-performance** MDS for large-scale DFSes.

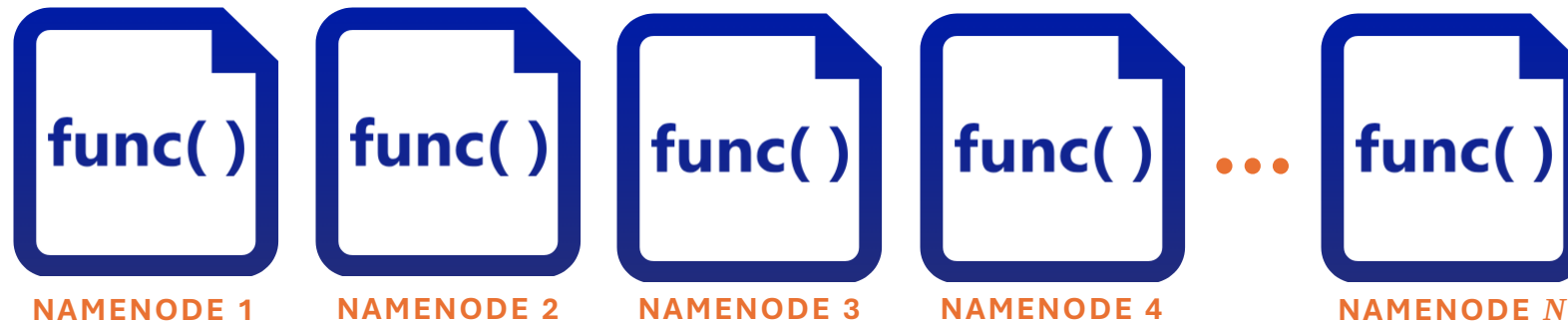


λFS: **Elastic**, **scalable**, and **high-performance** MDS for large-scale DFSes.

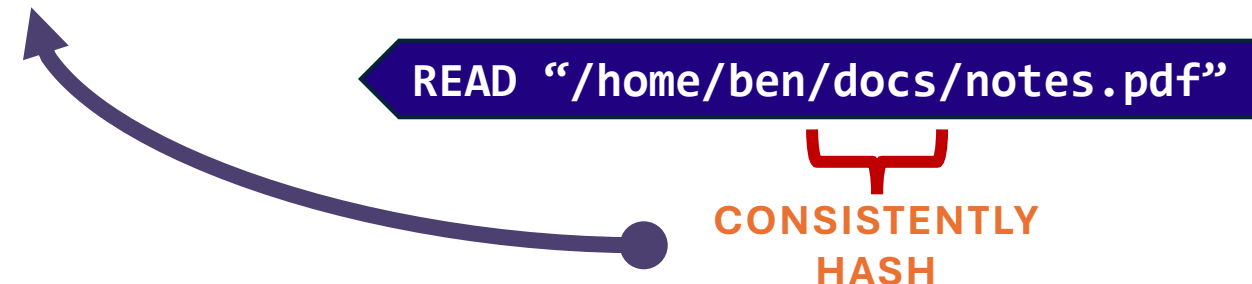




- λFS uses Apache OpenWhisk.
- **Elastic caching:** NameNodes (NNs) organized into N **function deployments**.



- **To partition namespace:**
Consistently hash on (unqualified) name
of *parent directory*.



KEY MECHANISM

Hybrid TCP-HTTP RPC Mechanism

λFS clients use a **hybrid TCP-HTTP RPC mechanism** to interact with NameNodes.

TCP

- Low latency (1 – 2 milliseconds).
- No resource scaling.
- Fault tolerance: λFS client library.

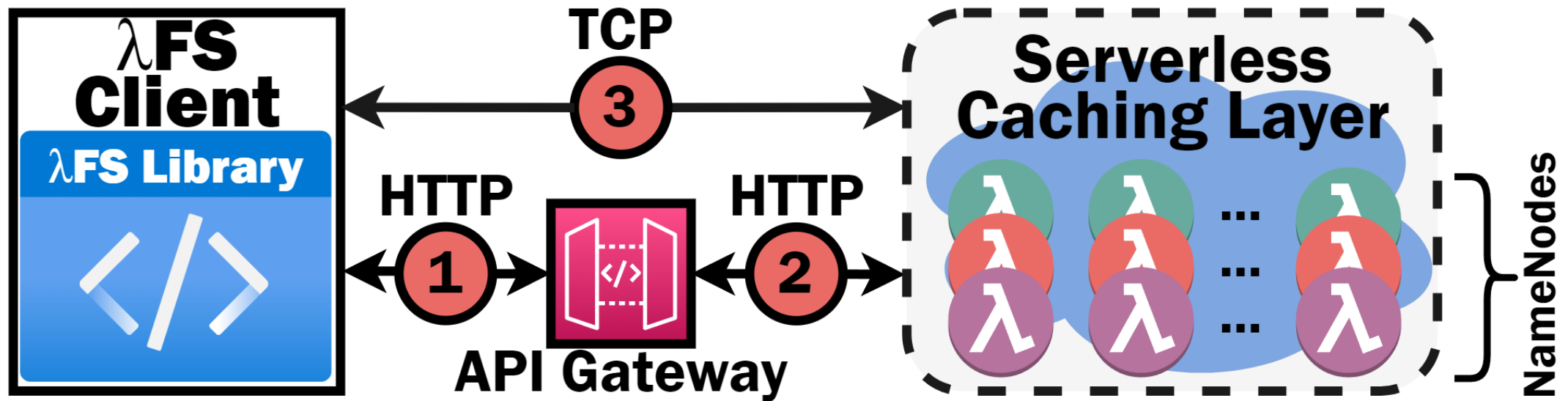
HTTP

- High latency (8 – 20 milliseconds).
- Built-in elastic resource scaling from FaaS platform.
- Built-in fault tolerance from FaaS platform.

Key: the **Hybrid TCP-HTTP RPC mechanism** enables the elasticity of the underlying FaaS platform to be **fine-tuned**

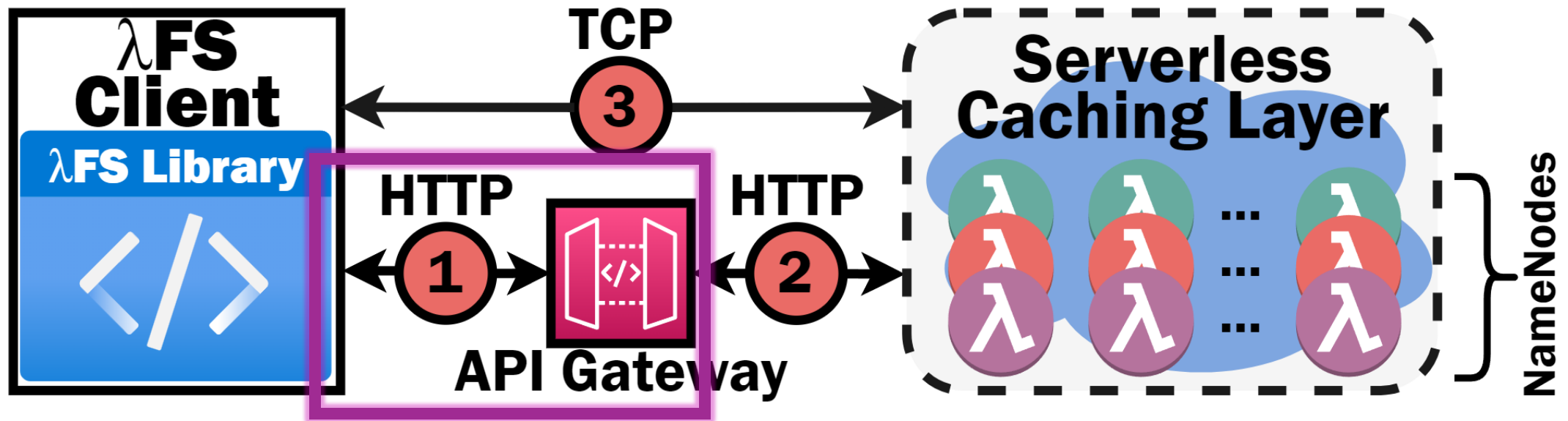
KEY MECHANISM

Hybrid TCP-HTTP RPC Mechanism



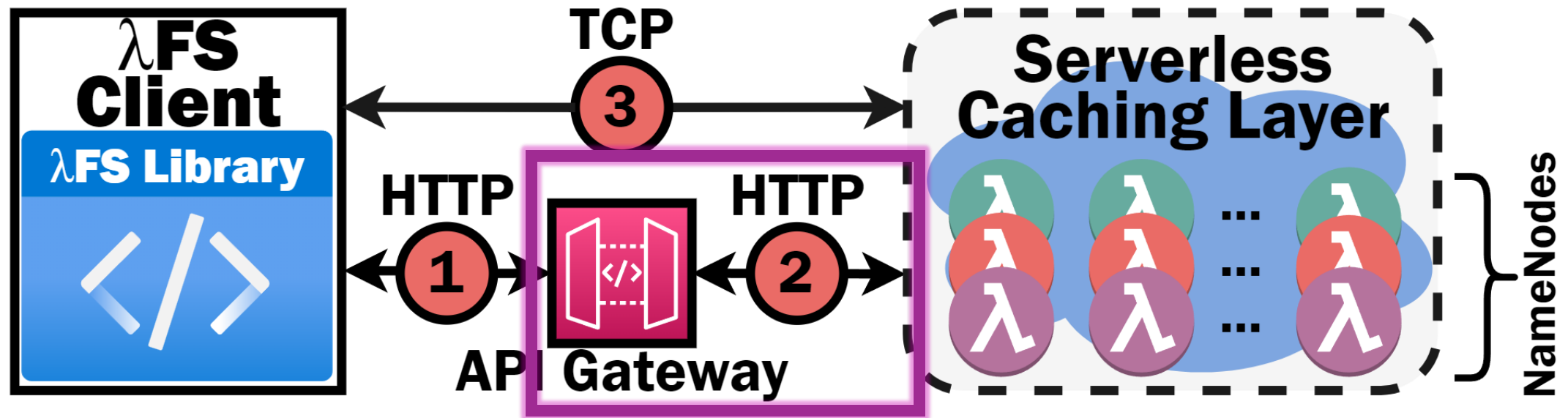
KEY MECHANISM

Hybrid TCP-HTTP RPC Mechanism



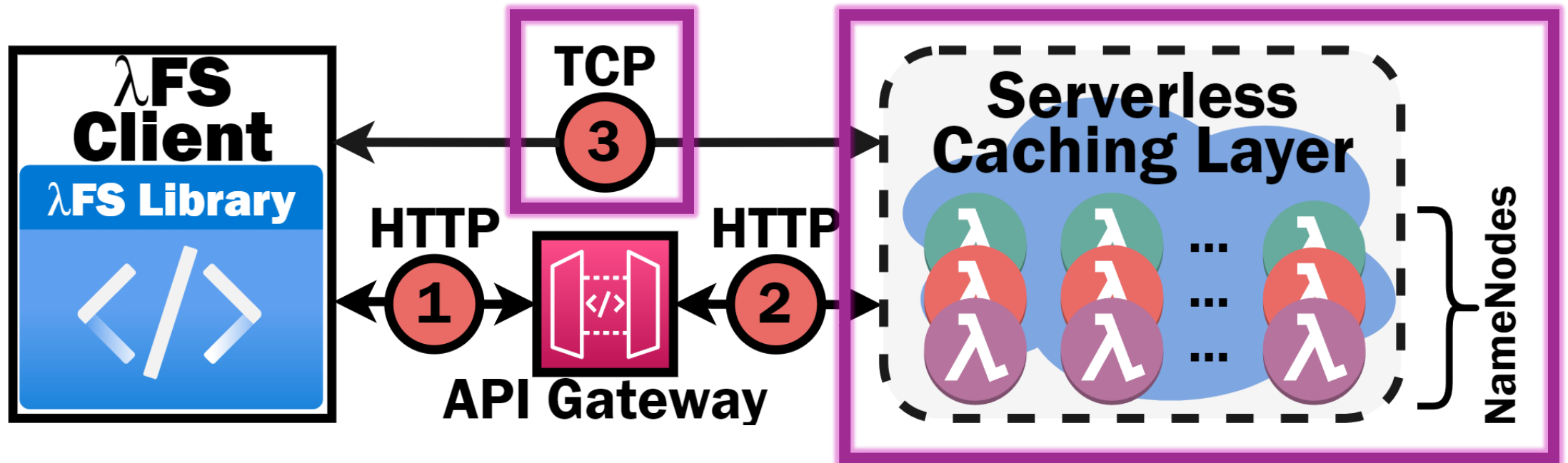
KEY MECHANISM

Hybrid TCP-HTTP RPC Mechanism



KEY MECHANISM

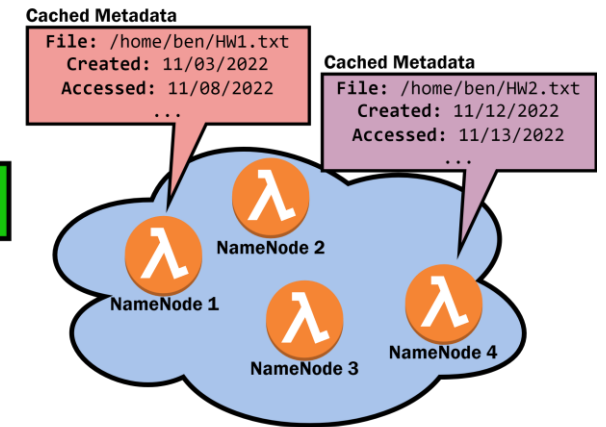
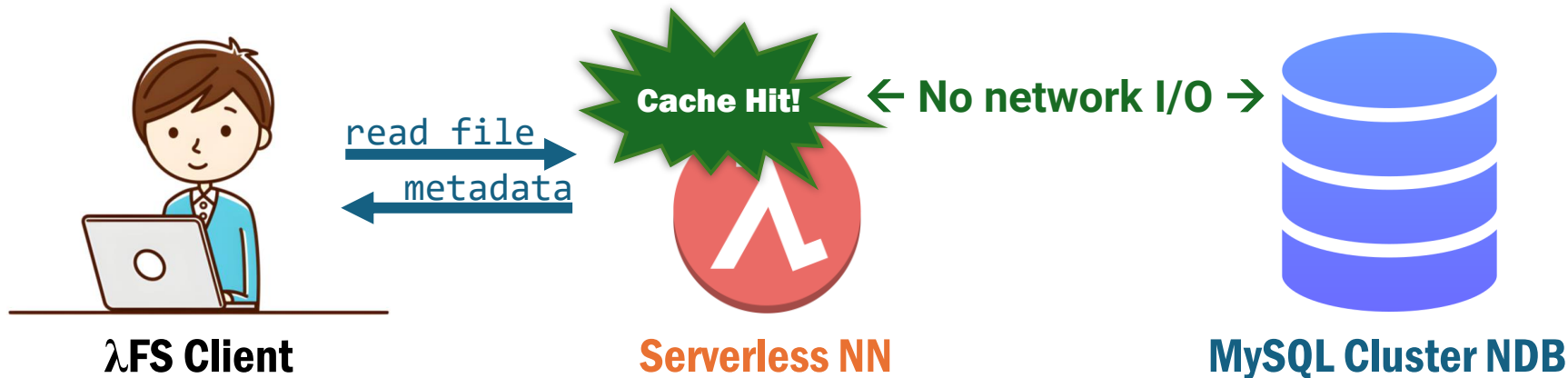
Hybrid TCP-HTTP RPC Mechanism



KEY MECHANISM

FaaS-Based Metadata Caching Layer

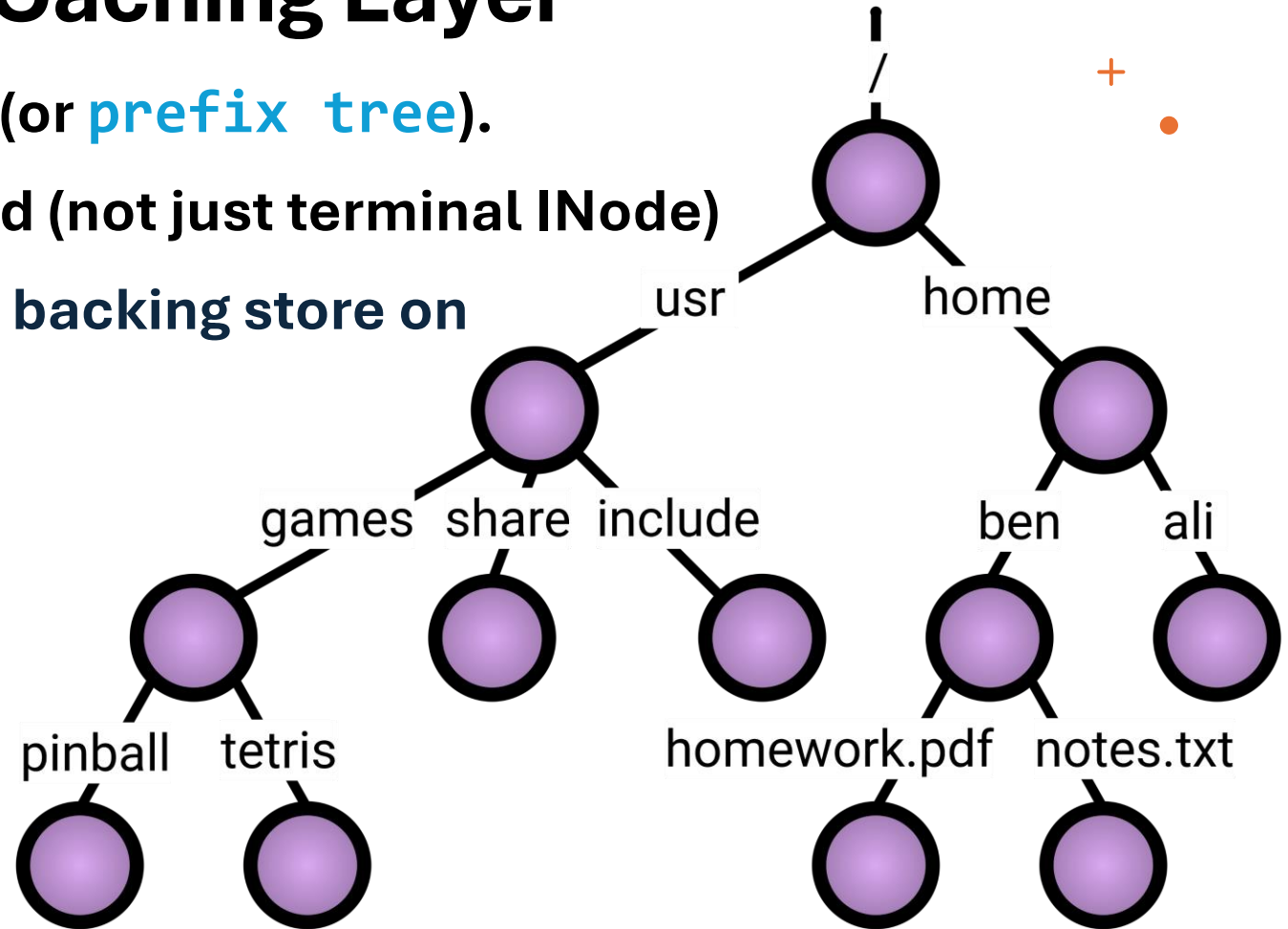
- Serverless NNs **cache metadata** in **main memory**
- Decreases per-operation latency ✓
- Improves overall system throughput ✓
- Reduces bottleneck of persistent backing store ✓



KEY MECHANISM

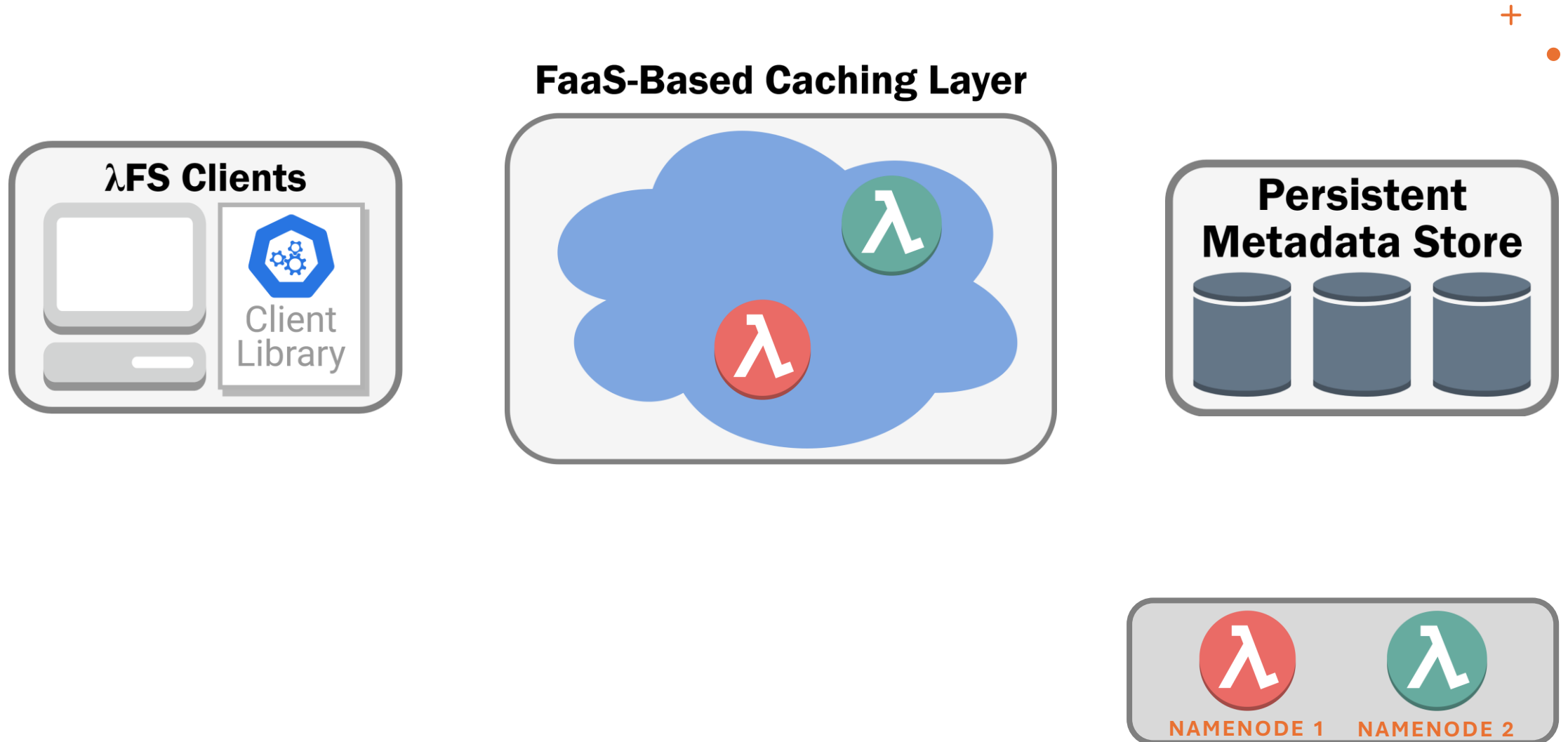
FaaS-Based Metadata Caching Layer

- Metadata stored within a **trie** (or **prefix tree**).
- **All metadata** in a path is cached (not just terminal INode)
- **Avoid** accessing the persistent backing store on **cache hits**



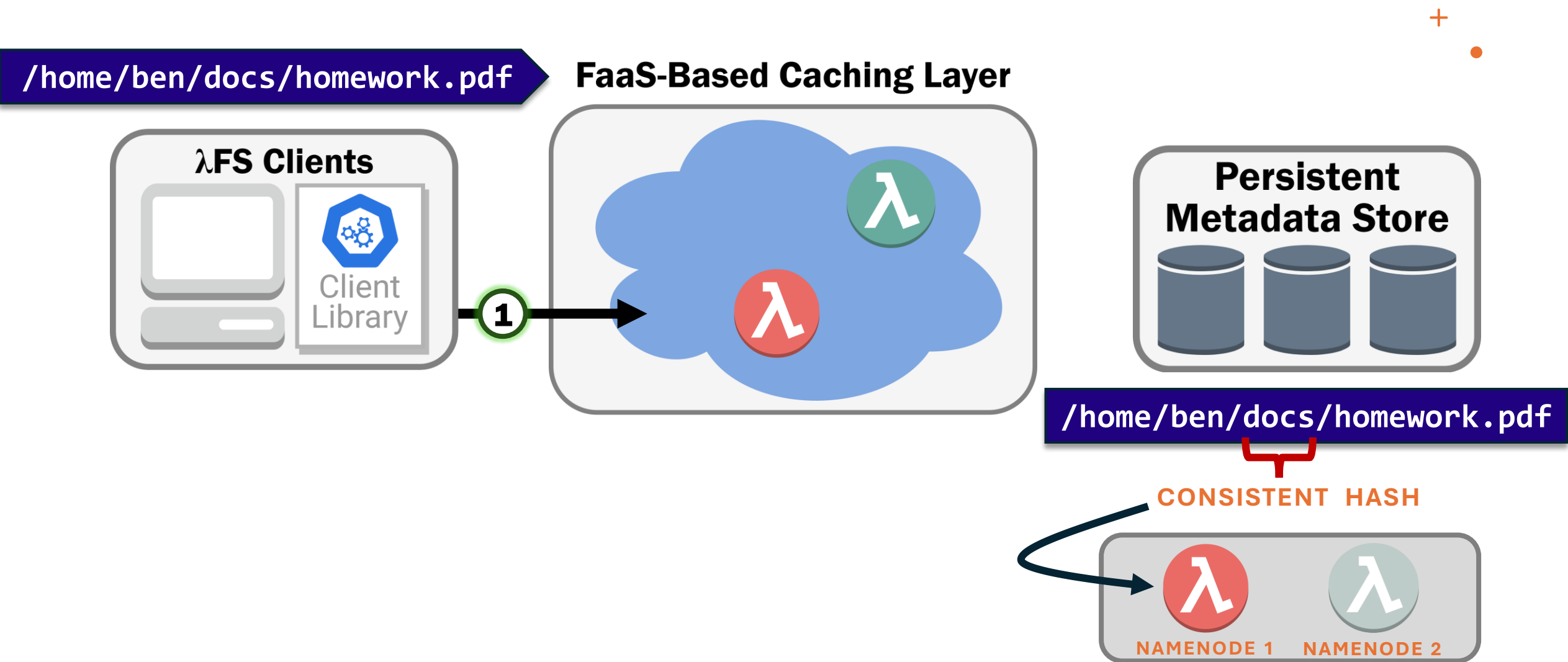
KEY MECHANISM

FaaS-Based Metadata Caching Layer



KEY MECHANISM

FaaS-Based Metadata Caching Layer



KEY MECHANISM

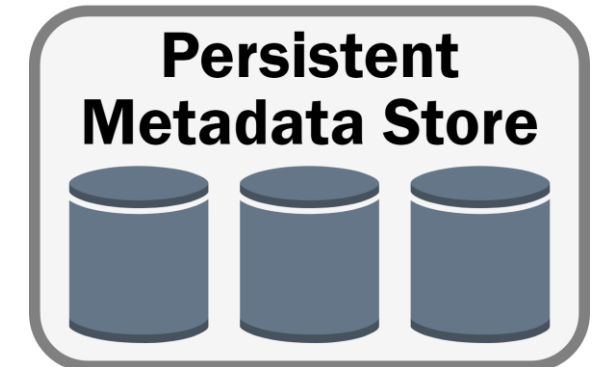
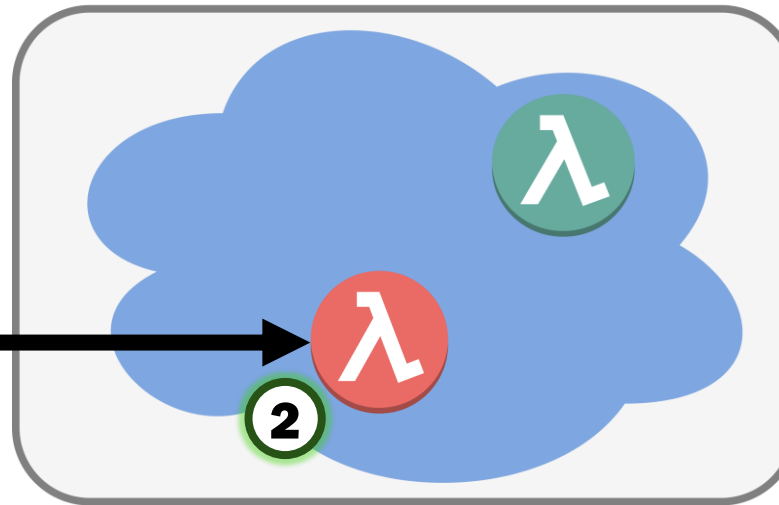
FaaS-Based Metadata Caching Layer

+

•

`/home/ben/docs/homework.pdf`

FaaS-Based Caching Layer



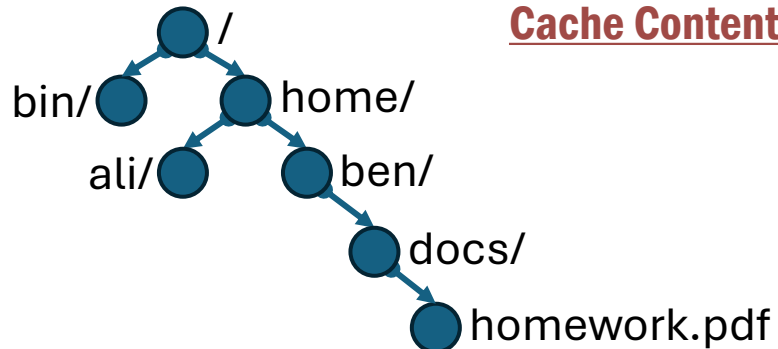
`/home/ben/docs/homework.pdf`



CONSISTENT HASH



Cache Contents

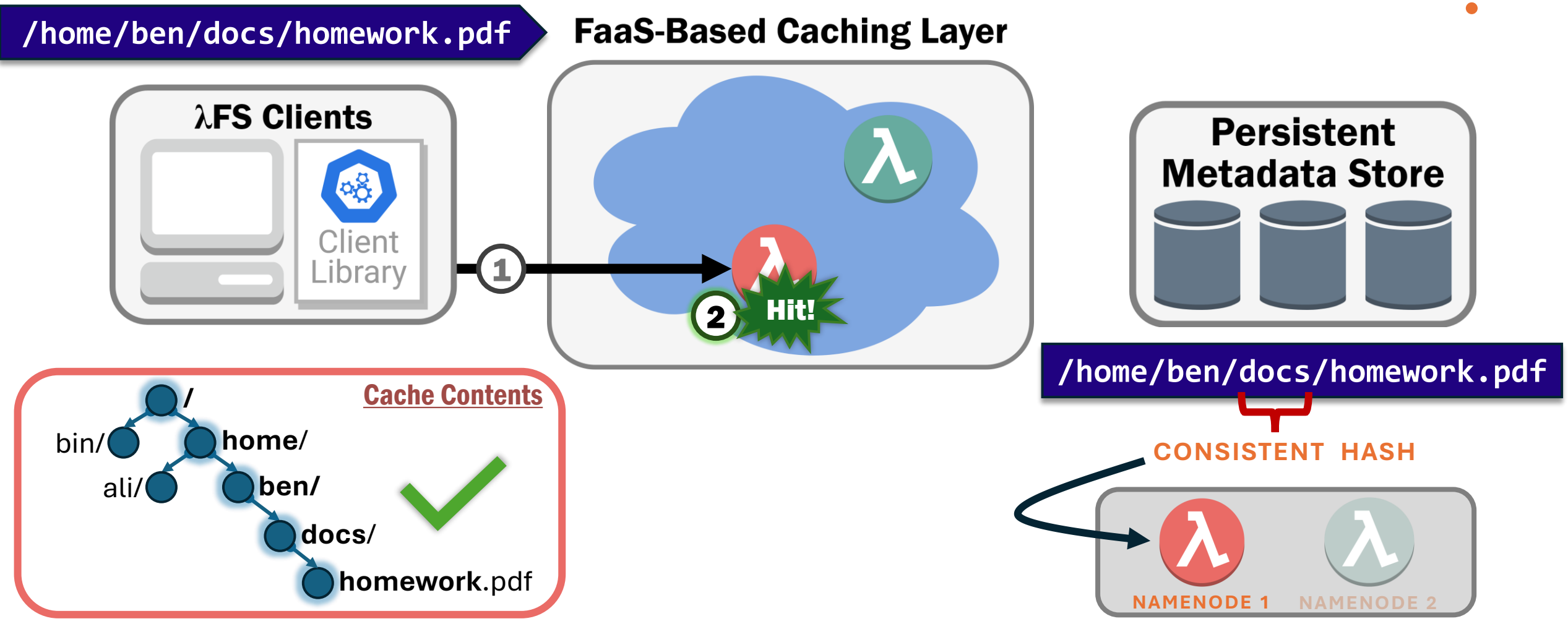


KEY MECHANISM

FaaS-Based Metadata Caching Layer

+

•



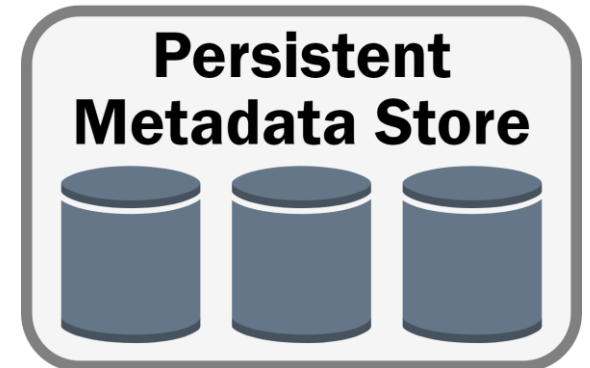
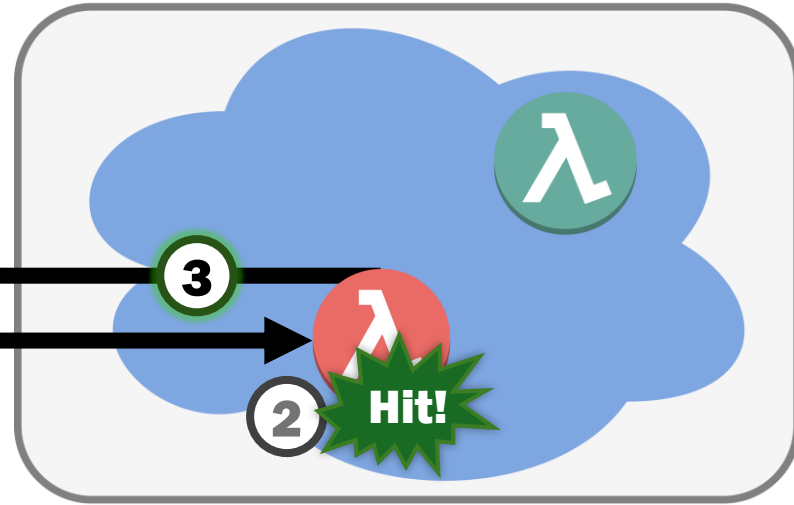
KEY MECHANISM

FaaS-Based Metadata Caching Layer



`/home/ben/docs/homework.pdf`

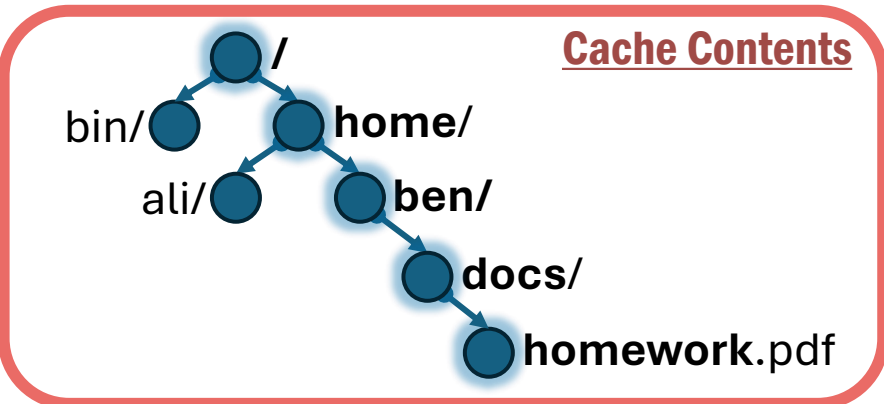
FaaS-Based Caching Layer



`/home/ben/docs/homework.pdf`

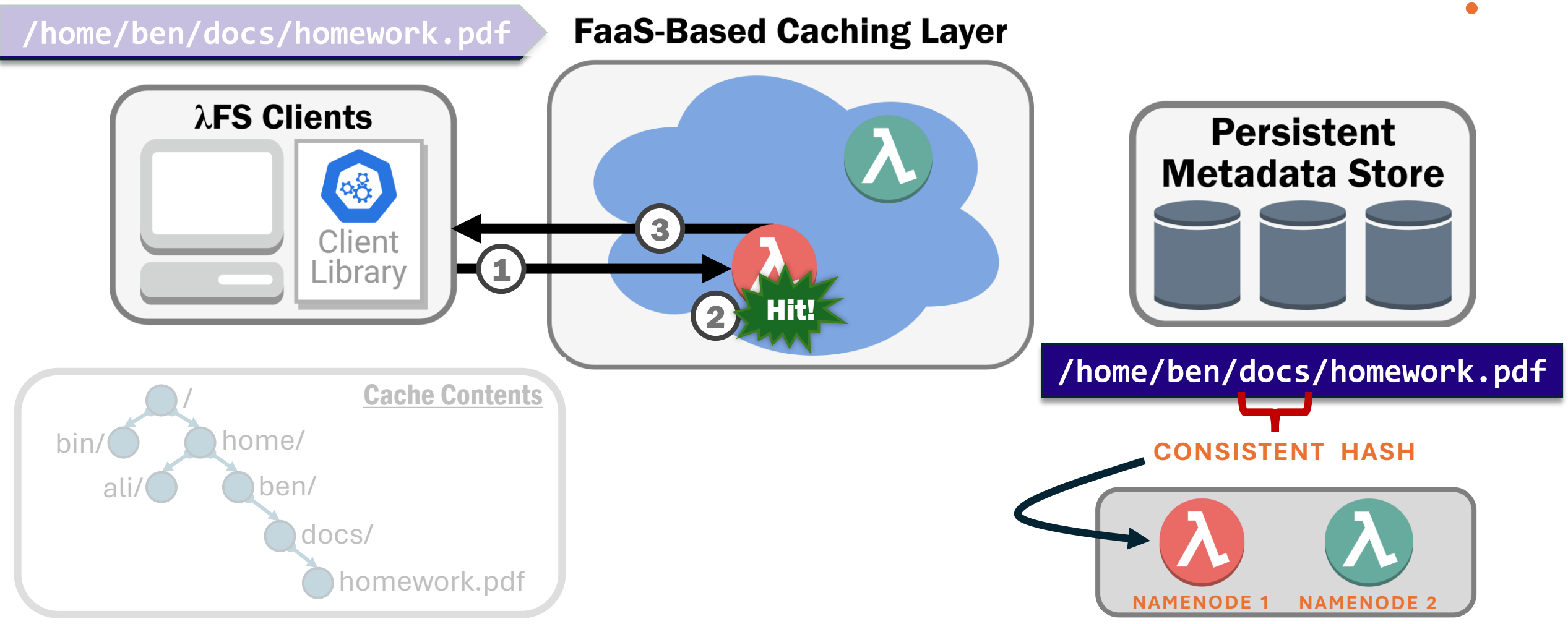


CONSISTENT HASH



KEY MECHANISM

FaaS-Based Metadata Caching Layer



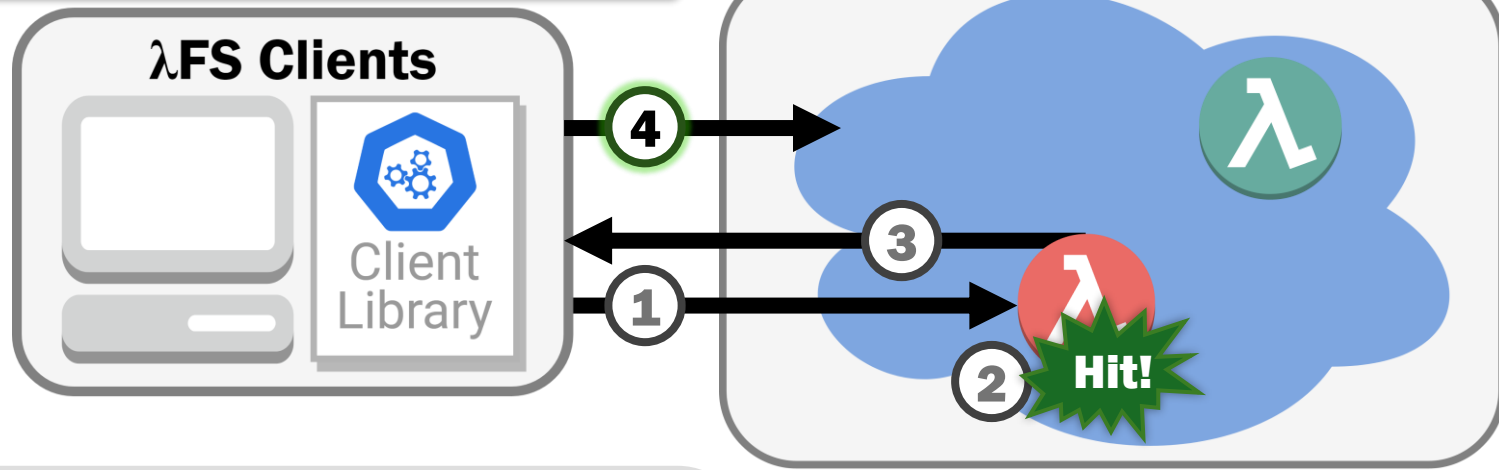
KEY MECHANISM

FaaS-Based Metadata Caching Layer

/home/ben/docs/homework.pdf

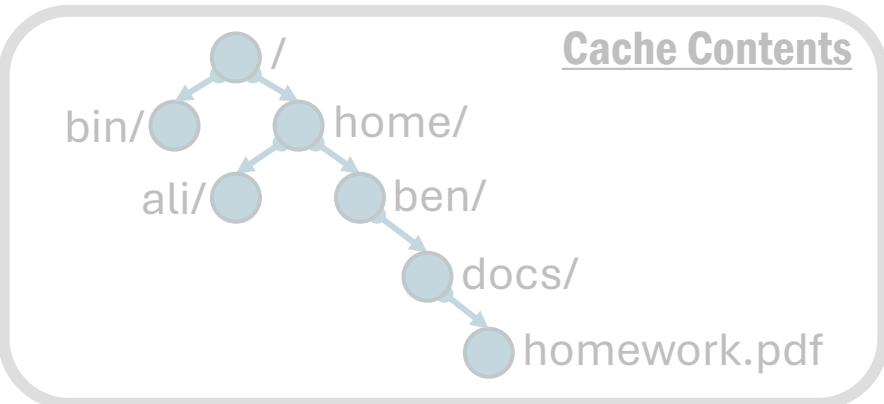
/home/ali/files/notes.txt

FaaS-Based Caching Layer



/home/ali/files/notes.txt

CONSISTENT HASH



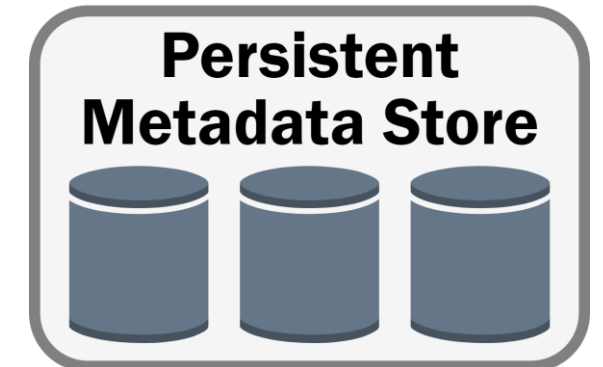
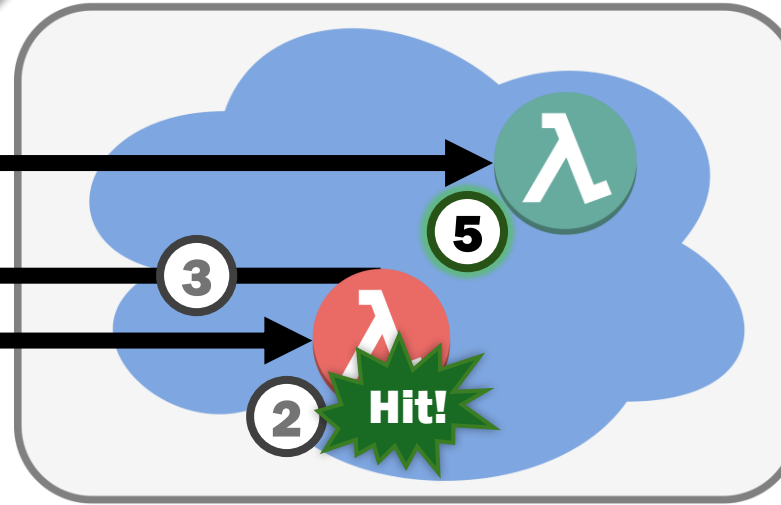
KEY MECHANISM

FaaS-Based Metadata Caching Layer

/home/ben/docs/homework.pdf

/home/ali/files/notes.txt

FaaS-Based Caching Layer

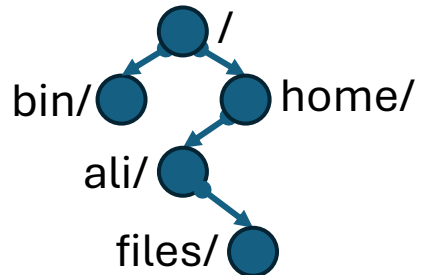


/home/ali/files/notes.txt

CONSISTENT HASH



Cache Contents



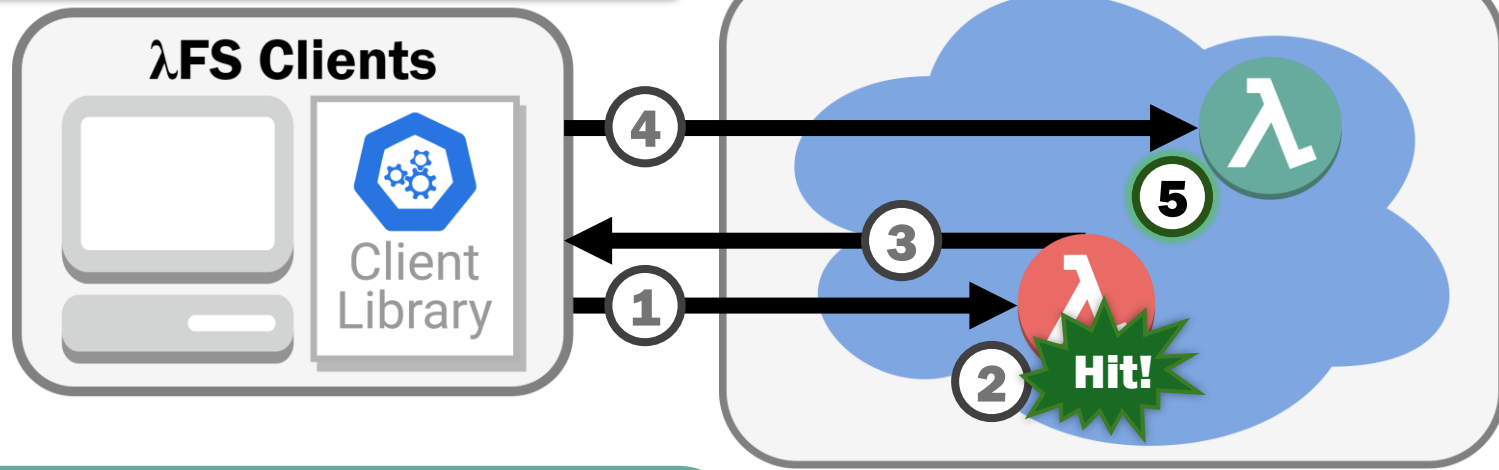
KEY MECHANISM

FaaS-Based Metadata Caching Layer

/home/ben/docs/homework.pdf

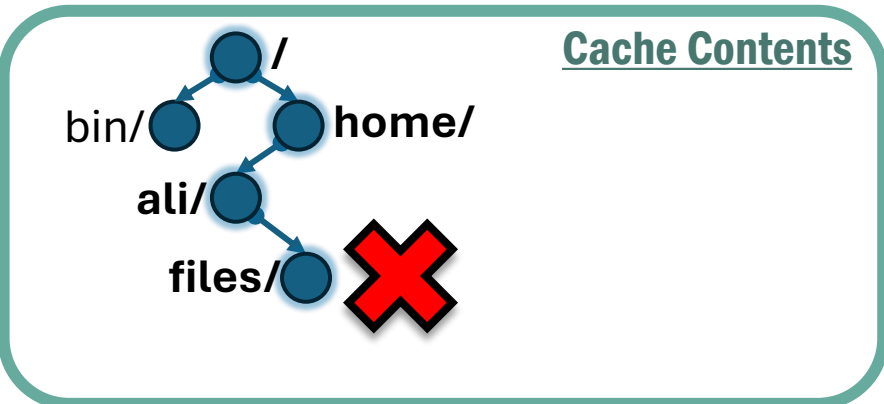
/home/ali/files/notes.txt

FaaS-Based Caching Layer



/home/ali/files/notes.txt

CONSISTENT HASH



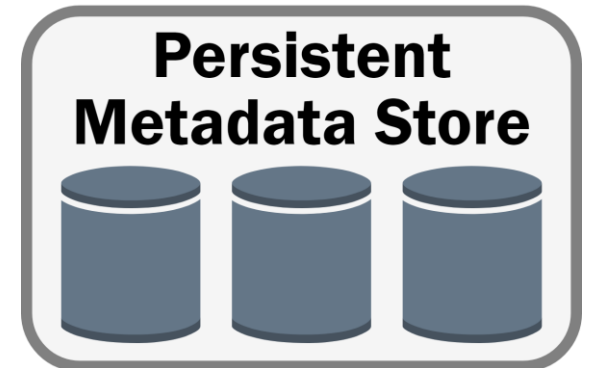
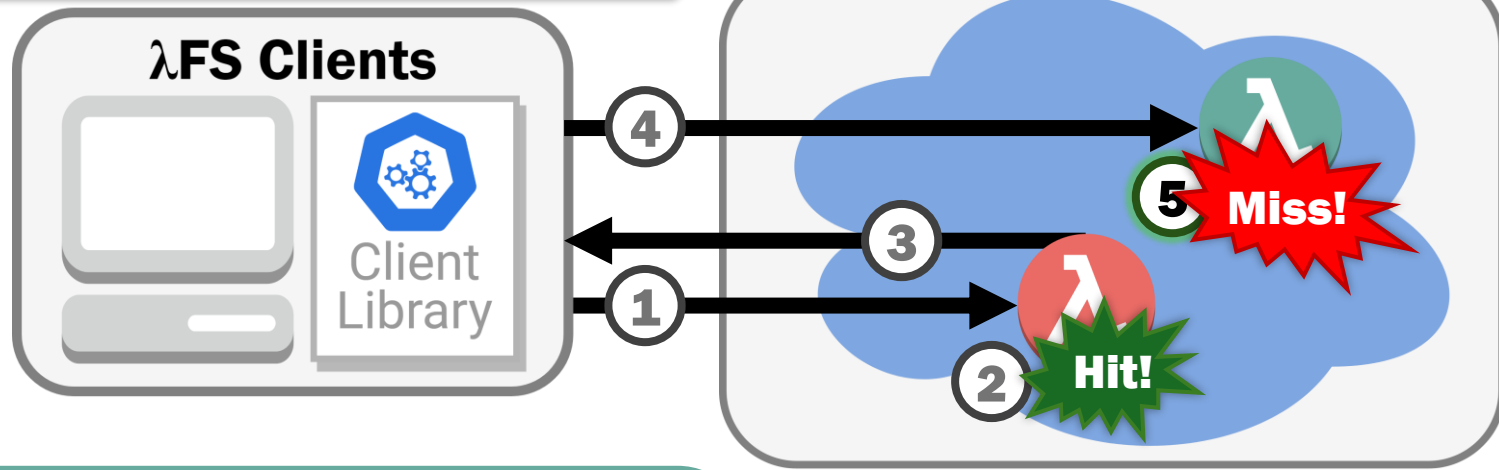
KEY MECHANISM

FaaS-Based Metadata Caching Layer

/home/ben/docs/homework.pdf

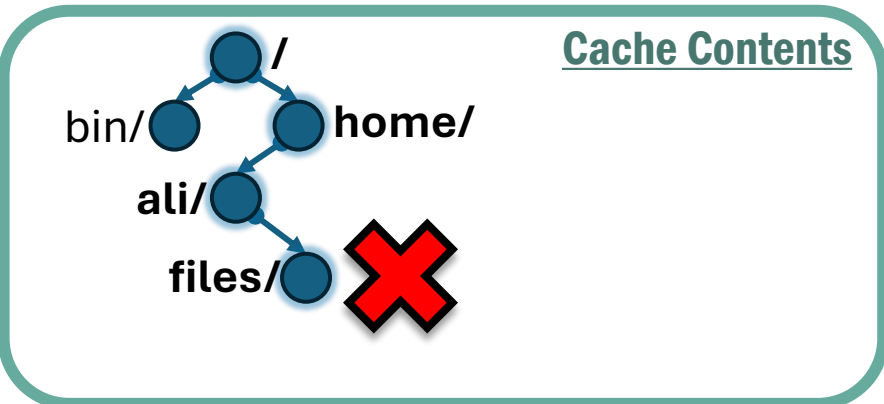
/home/ali/files/notes.txt

FaaS-Based Caching Layer



/home/ali/files/notes.txt

CONSISTENT HASH



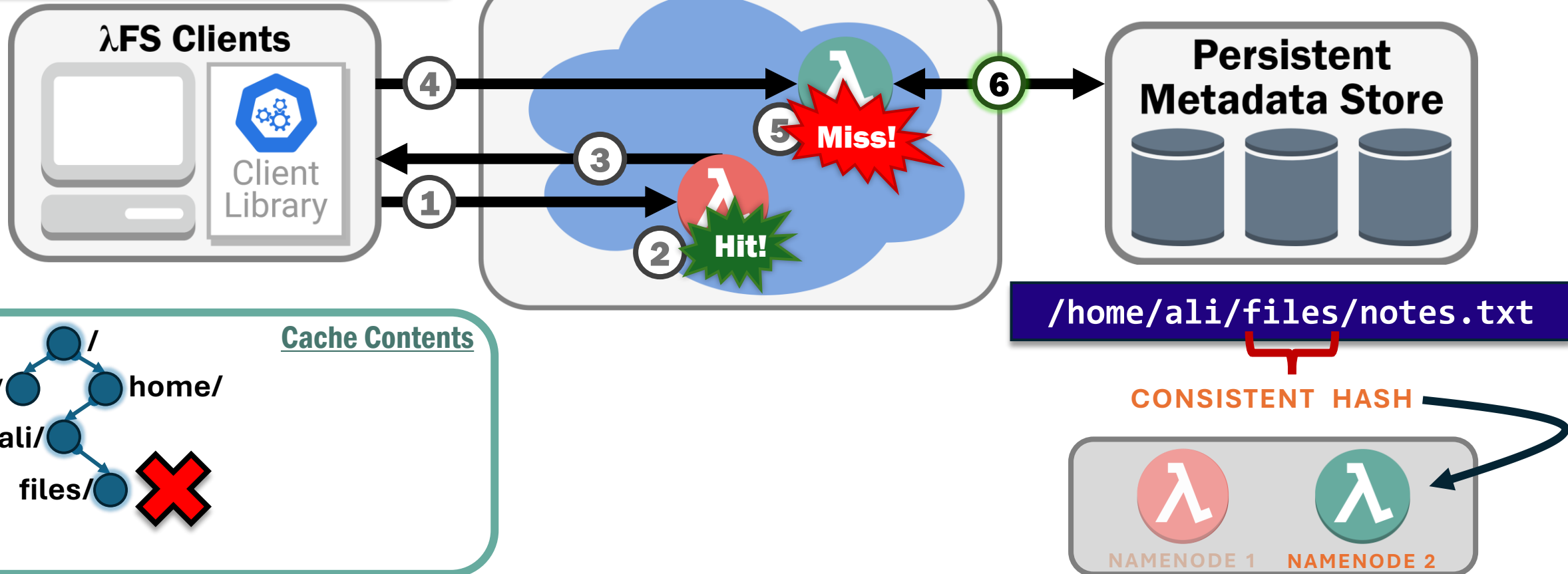
KEY MECHANISM

FaaS-Based Metadata Caching Layer

/home/ben/docs/homework.pdf

/home/ali/files/notes.txt

FaaS-Based Caching Layer



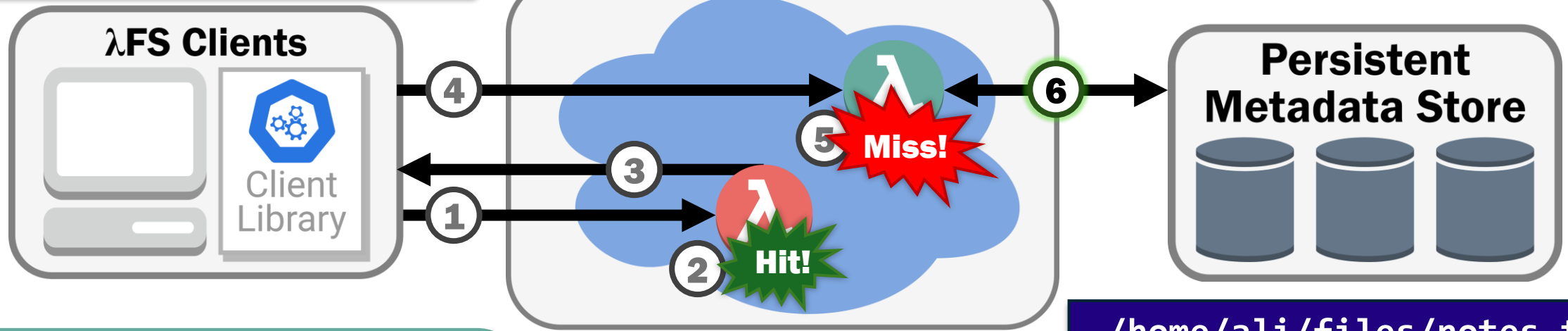
KEY MECHANISM

FaaS-Based Metadata Caching Layer

/home/ben/docs/homework.pdf

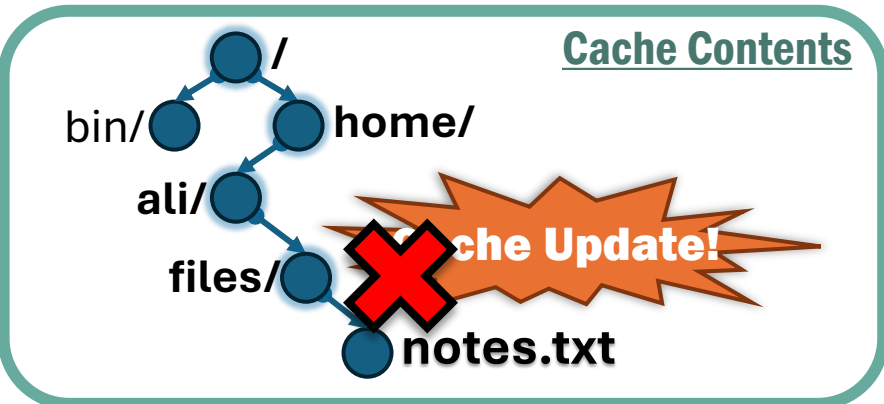
/home/ali/files/notes.txt

FaaS-Based Caching Layer



/home/ali/files/notes.txt

CONSISTENT HASH



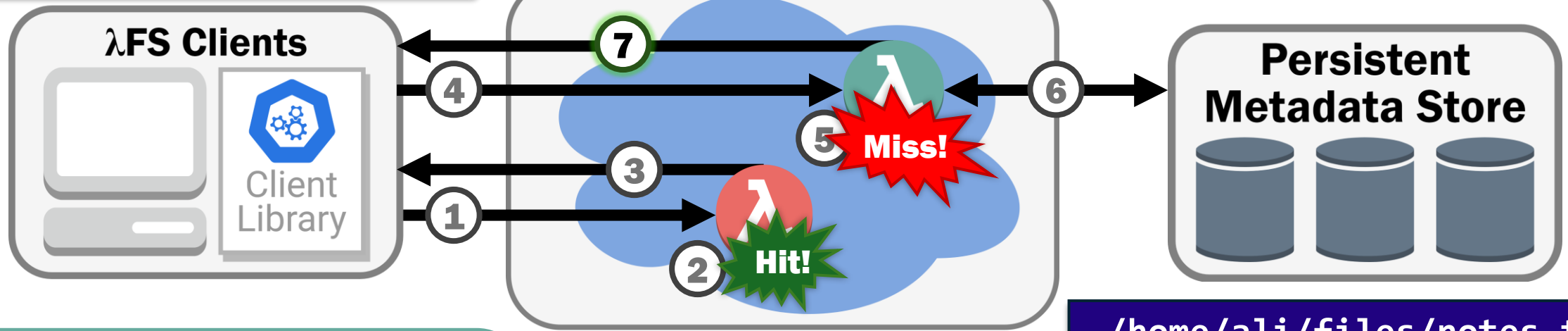
KEY MECHANISM

FaaS-Based Metadata Caching Layer

/home/ben/docs/homework.pdf

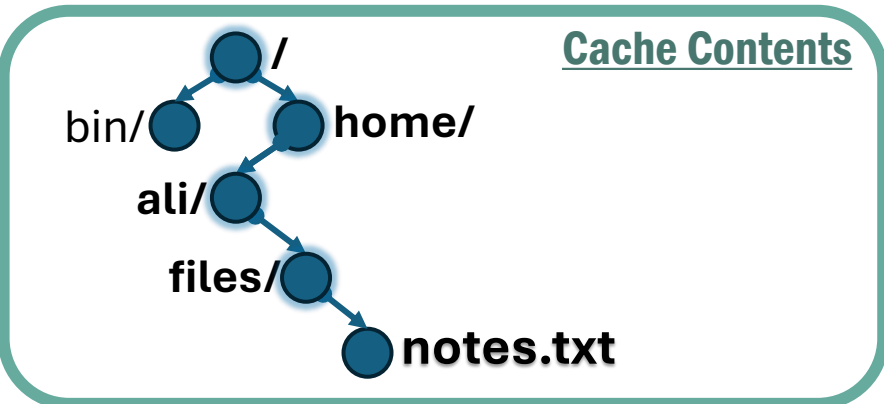
/home/ali/files/notes.txt

FaaS-Based Caching Layer



/home/ali/files/notes.txt

CONSISTENT HASH

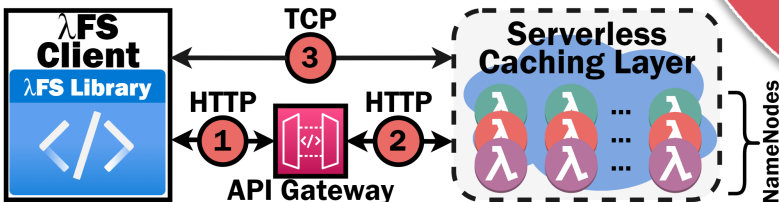
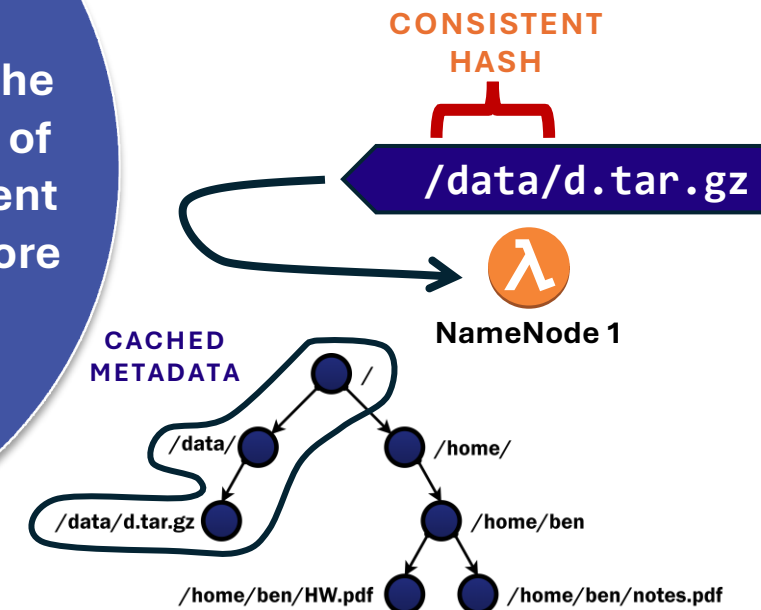
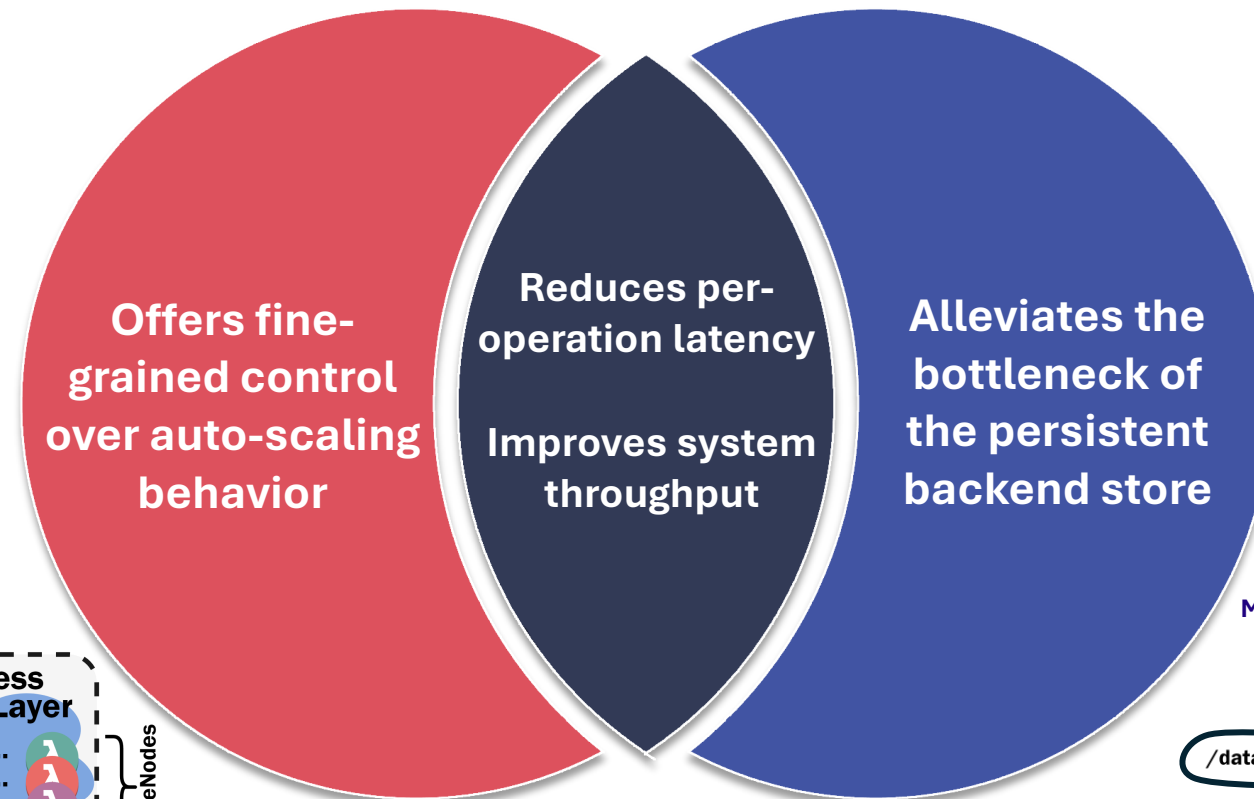


λFS Design Recap

λFS: an **elastic**, **scalable**, & **high-performance** MDS for large-scale DFSes

Hybrid HTTP-TCP RPC Mechanism

FaaS-Based Metadata Cache



λFS Evaluation

Real-World, Industrial Workload

- Randomly-generated workload based on **Spotify's HDFS cluster trace**.
- Evaluated auto-scaling policy via **request bursts (up to 7x base throughput)**.

Operation	Percentage	Operation	Percentage
create file	2.7%	read file	69.22%
mkdirs	0.02%	stat file/dir	17%
delete file/dir	0.75%	list file/dir	9.01%
mv file/dir	1.3%	Total Read Ops	95.23%

+

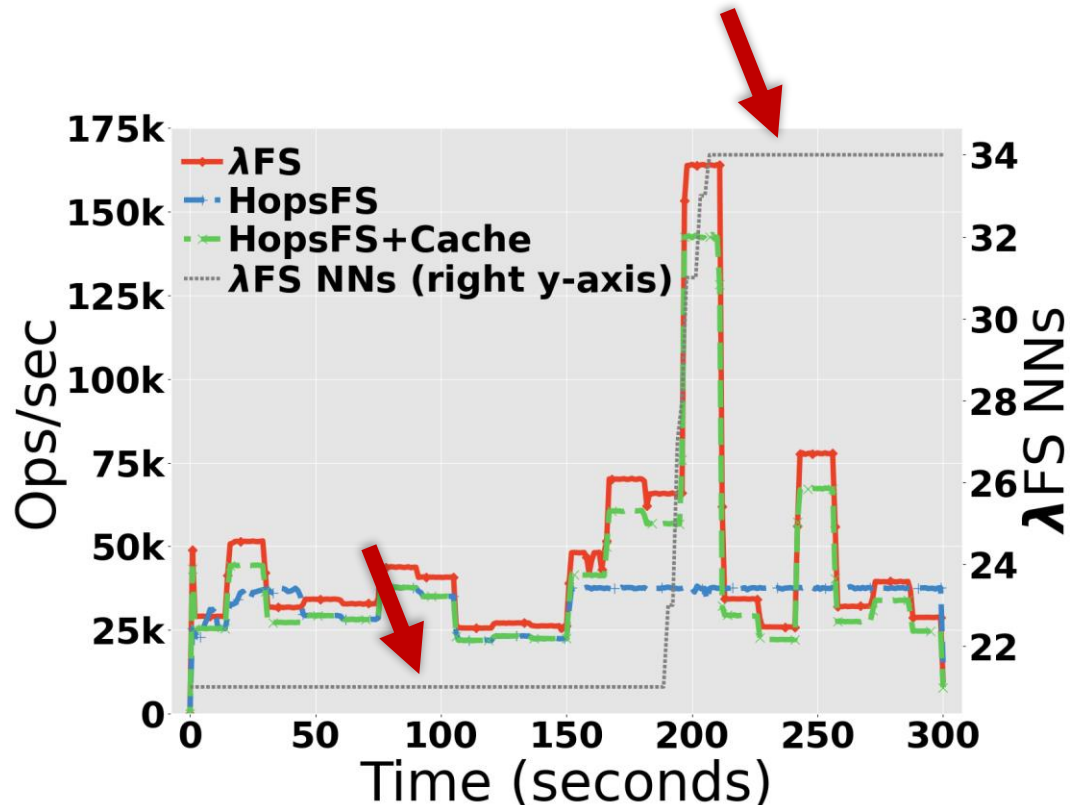


λFS Evaluation

Real-World, Industrial Workload

- Randomly-generated workload based on **Spotify's HDFS cluster trace**.
- Evaluated auto-scaling policy via **request bursts** (up to **7x base throughput**).

Operation	Percentage	Operation	Percentage
create file	2.7%	read file	69.22%
mkdirs	0.02%	stat file/dir	17%
delete file/dir	0.75%	list file/dir	9.01%
mv file/dir	1.3%	Total Read Ops	95.23%



λFS Evaluation

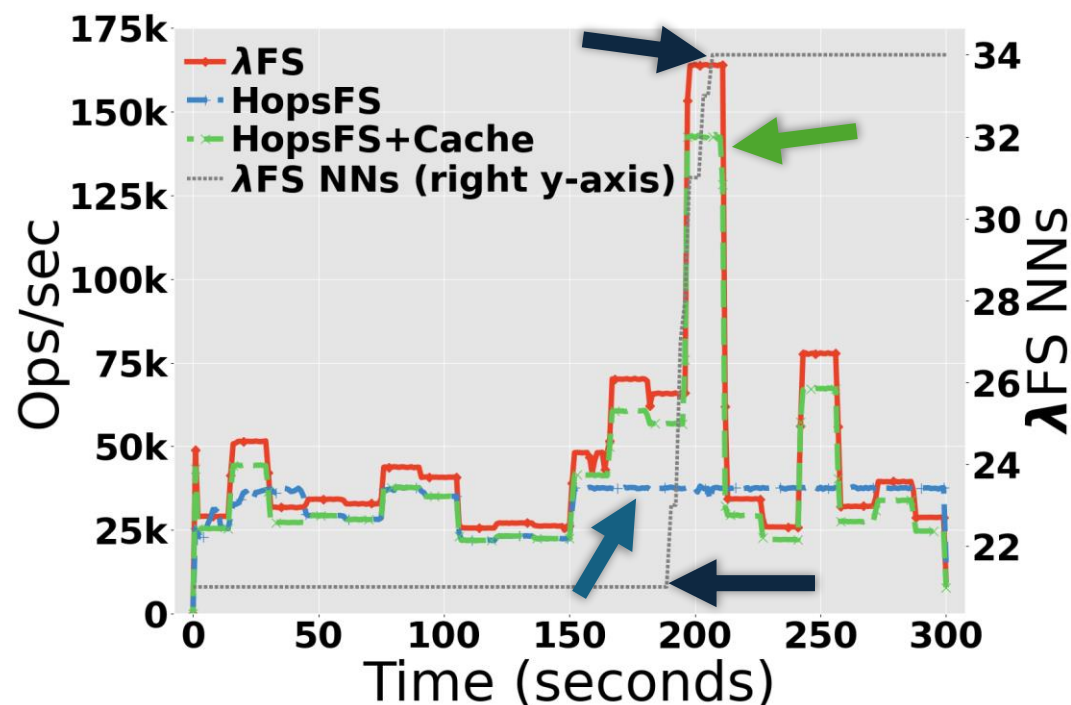
Real-World, Industrial Workload

- Randomly-generated workload based on **Spotify's HDFS cluster trace**.
- Evaluated auto-scaling policy via **request bursts (up to 7x base throughput)**.

Base throughput: 25,000 ops/sec

HopsFS: 512 vCPU **λFS:** 256 vCPU

Operation	Percentage	Operation	Percentage
create file	2.7%	read file	69.22%
mkdirs	0.02%	stat file/dir	17%
delete file/dir	0.75%	list file/dir	9.01%
mv file/dir	1.3%	Total Read Ops	95.23%

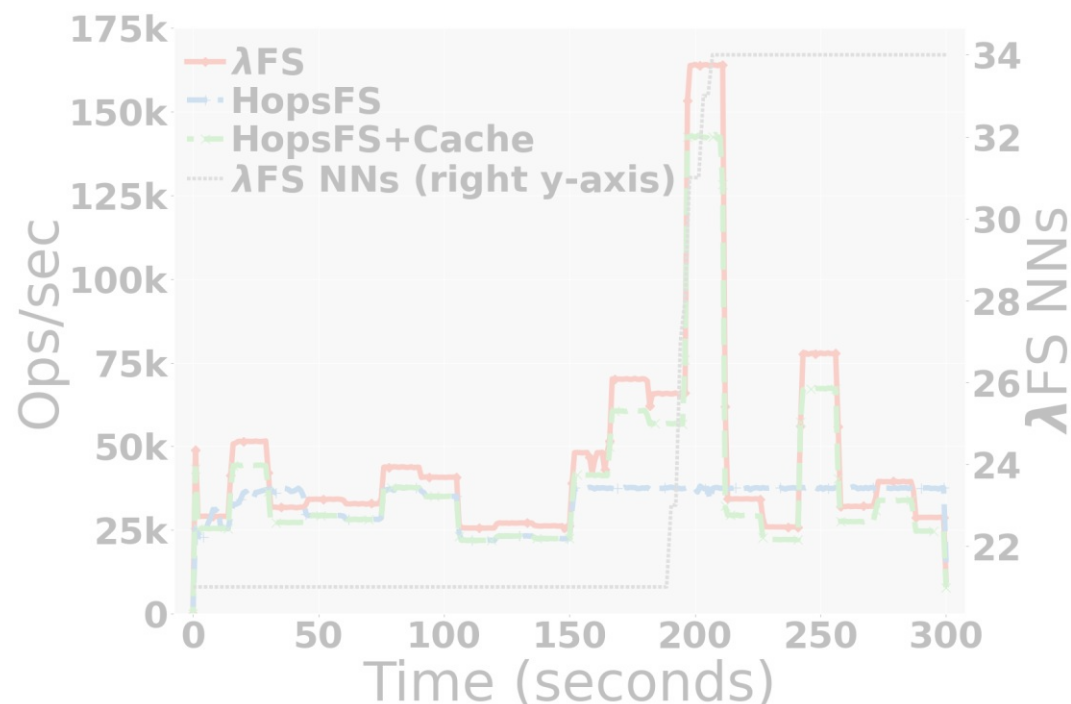


λFS Evaluation

Real-World, Industrial Workload

- Randomly-generated workload based on **Spotify's HDFS cluster trace**.
- Evaluated auto-scaling policy via **request bursts** (up to **7x base throughput**).
- Evaluated cost-efficiency via **performance-per-cost** metric: $\frac{\text{Instantaneous Throughput}}{\text{Instantaneous Cost}}$

Operation	Percentage	Operation	Percentage
create file	2.7%	read file	69.22%
mkdirs	0.02%	stat file/dir	17%
delete file/dir	0.75%	list file/dir	9.01%
mv file/dir	1.3%	Total Read Ops	95.23%

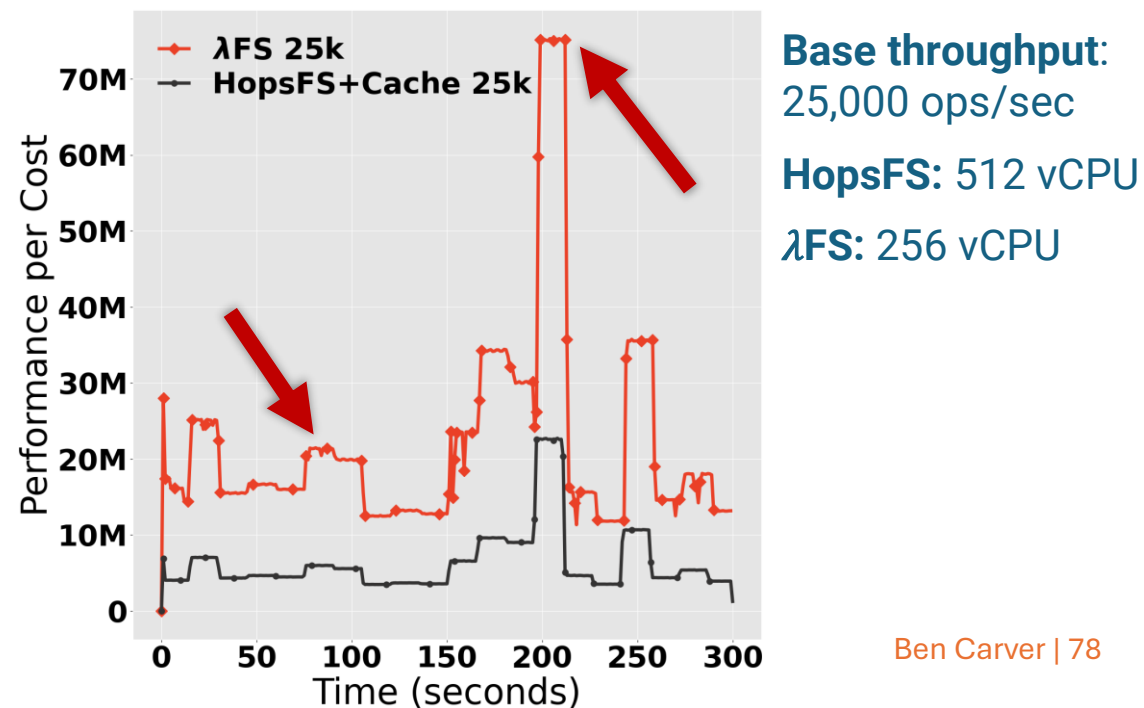
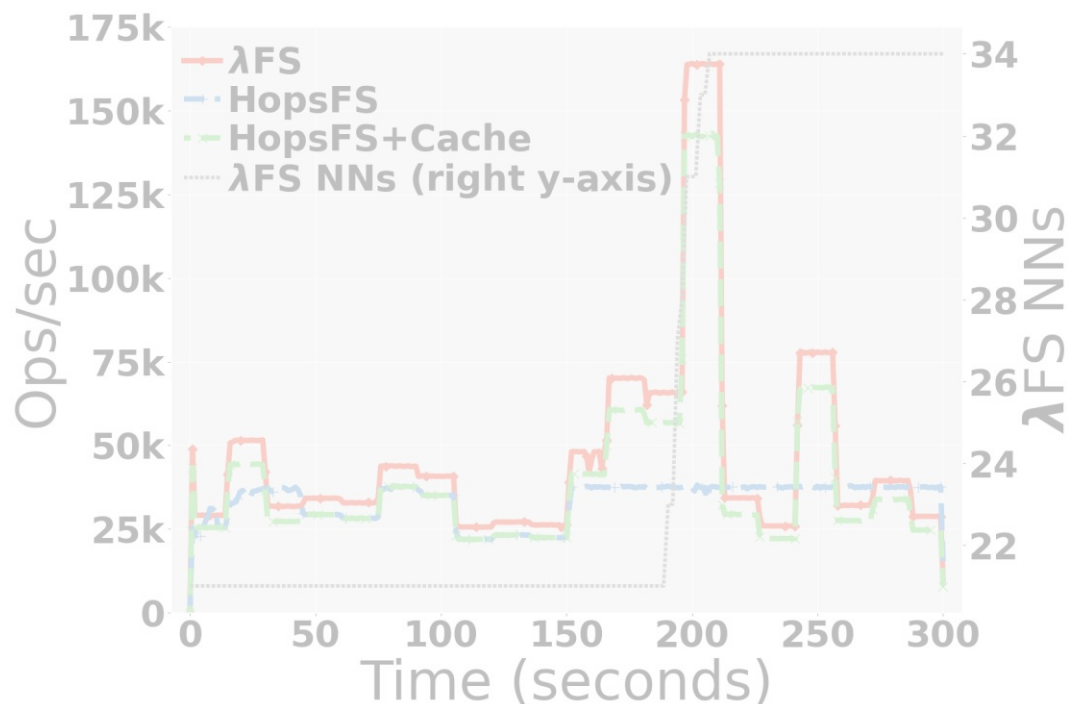


λFS Evaluation

Real-World, Industrial Workload

- Randomly-generated workload based on **Spotify's HDFS cluster trace**.
- Evaluated auto-scaling policy via **request bursts** (up to **7x base throughput**).
- Evaluated cost-efficiency via **performance-per-cost** metric: **Instantaneous Throughput**
Instantaneous Cost

Operation	Percentage	Operation	Percentage
create file	2.7%	read file	69.22%
mkdirs	0.02%	stat file/dir	17%
delete file/dir	0.75%	list file/dir	9.01%
mv file/dir	1.3%	Total Read Ops	95.23%



λFS Evaluation

Client-Driven Scaling

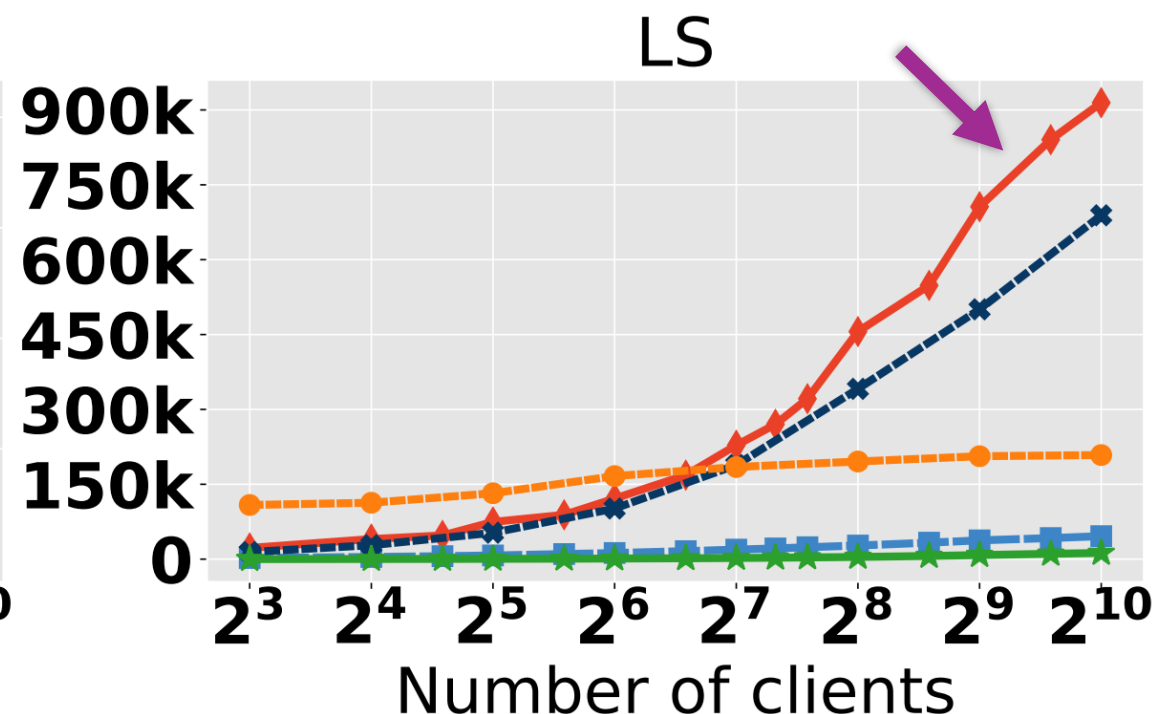
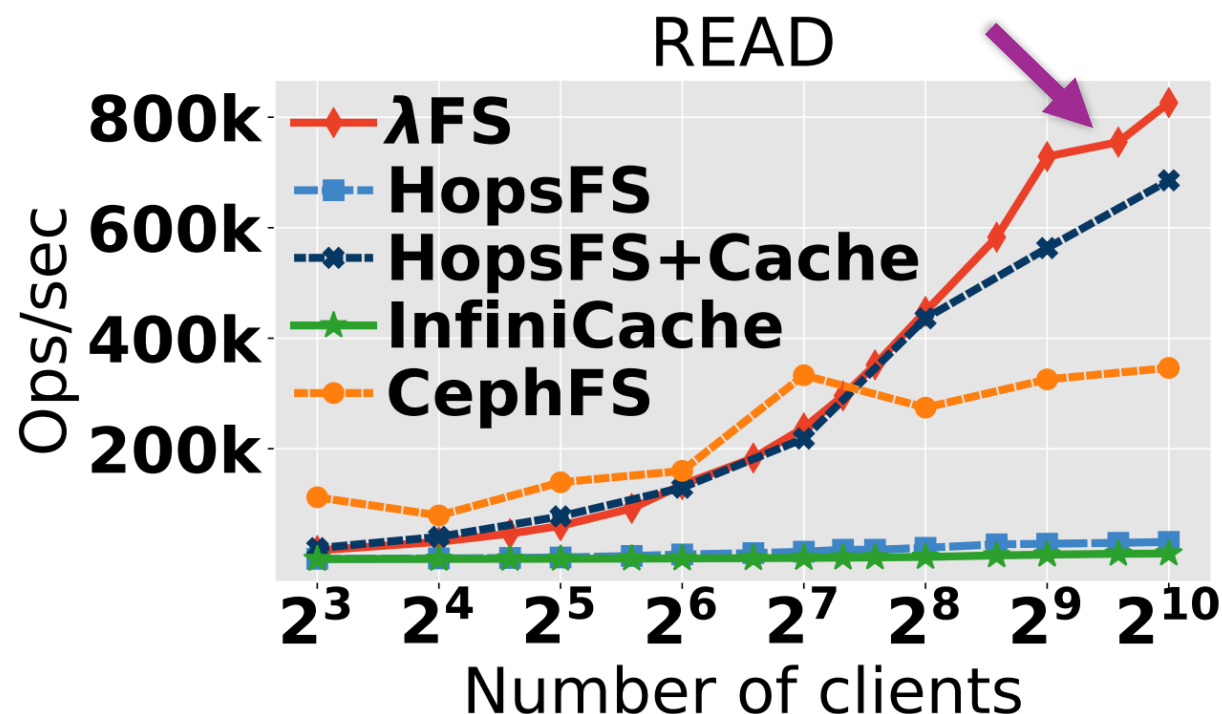
- Varied number of clients from $2^3 - 2^{10}$ (i.e., 8 – 1,024); each performed 3,072 FS operations.



λFS Evaluation

Client-Driven Scaling

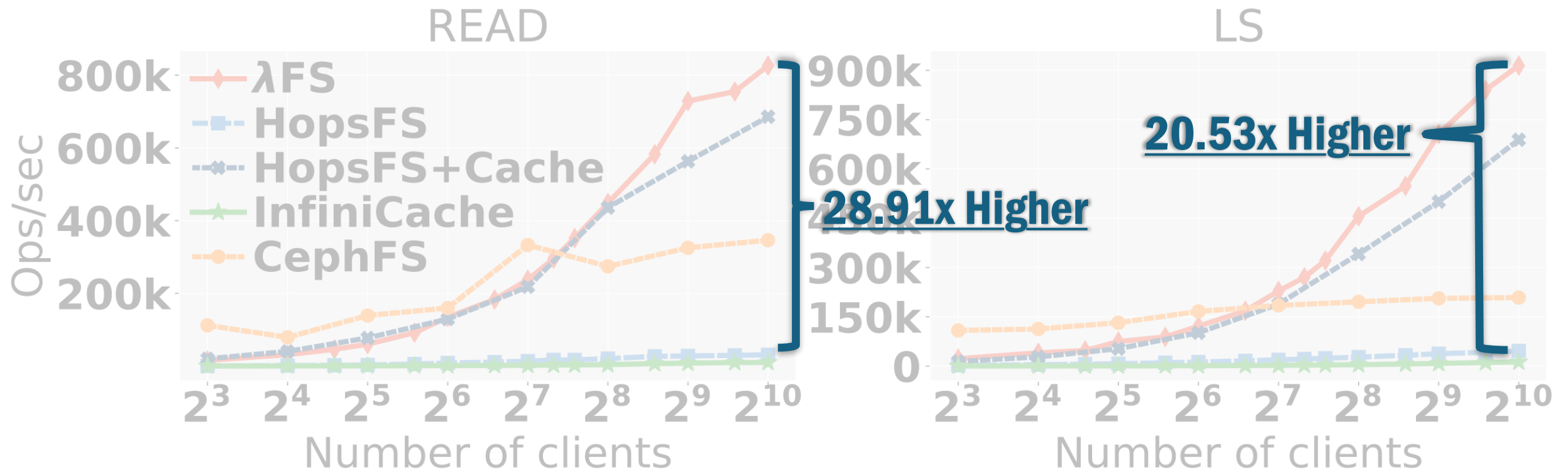
- Varied number of clients from $2^3 - 2^{10}$ (i.e., 8 – 1,024); each performed 3,072 FS operations.
- Aggregated throughput plotted on the y-axis.



λFS Evaluation

Client-Driven Scaling

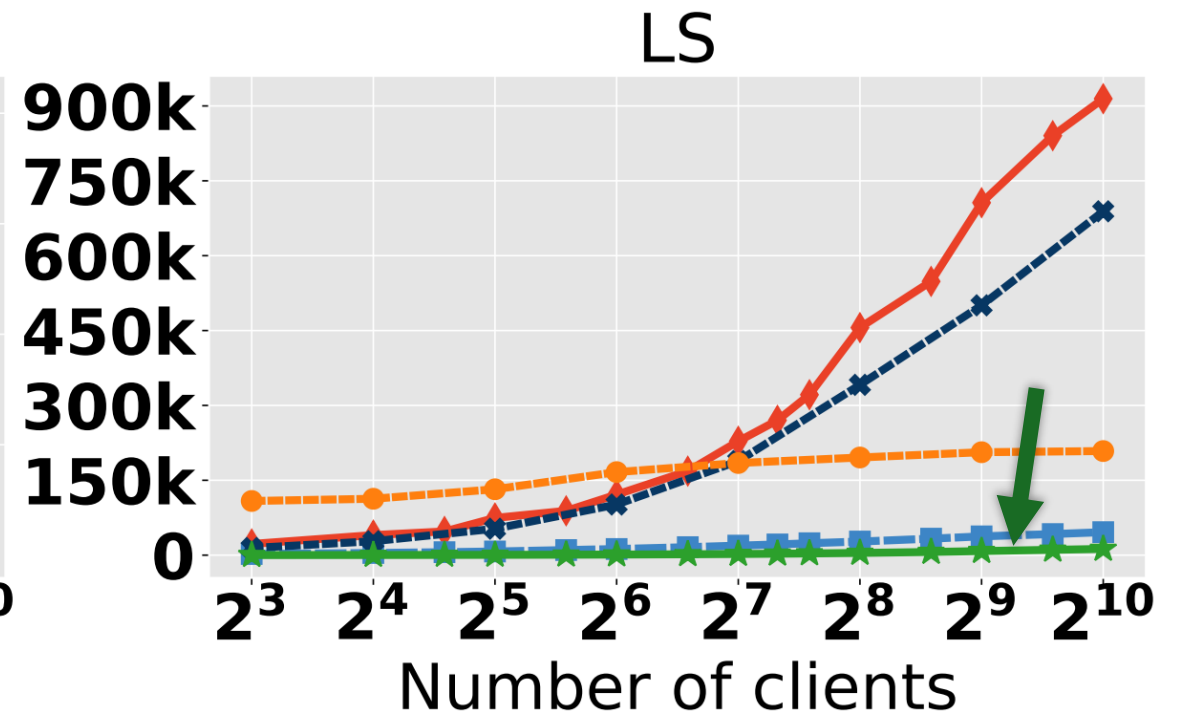
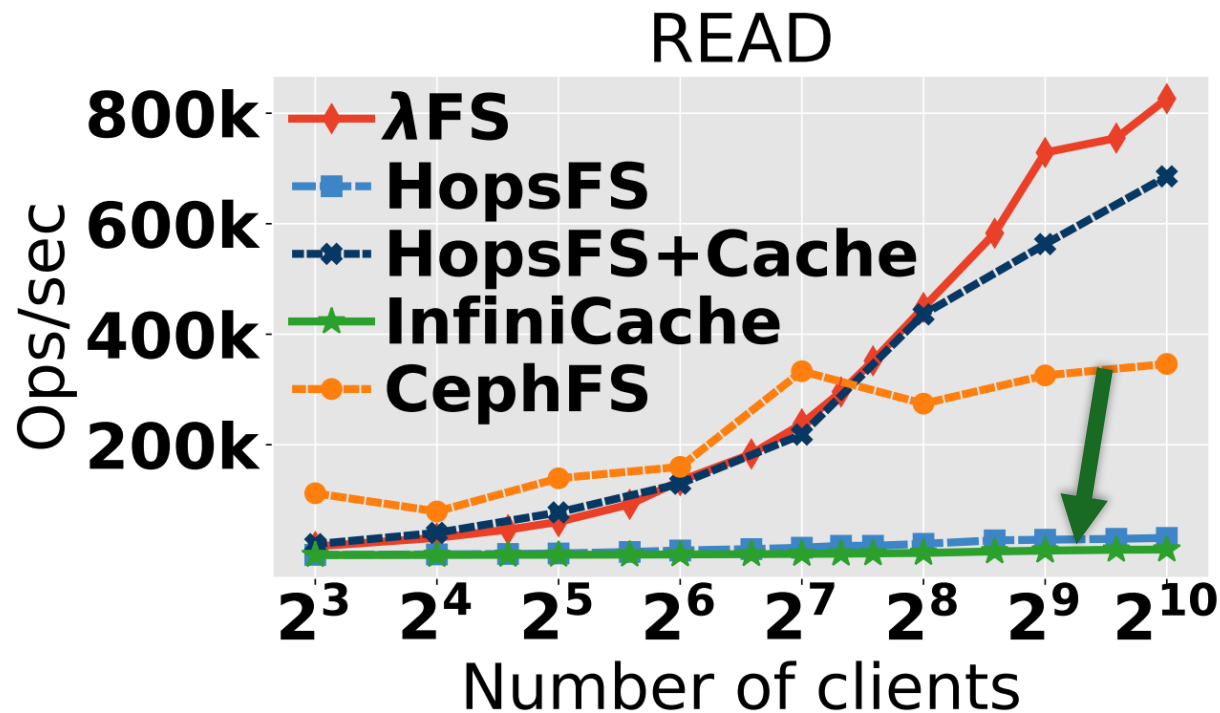
- Varied number of clients from $2^3 - 2^{10}$ (i.e., 8 – 1,024); each performed 3,072 FS operations.
- Aggregated throughput plotted on the y-axis.



λFS Evaluation

Client-Driven Scaling

- Varied number of clients from $2^3 - 2^{10}$ (i.e., 8 – 1,024); each performed 3,072 FS operations.
- Aggregated throughput plotted on the y-axis.



λFS Conclusion

- ❖ First **cloud-native** DFS MDS using **FaaS** to cache and elastically scale DFS metadata.
- ❖ Achieves **high-throughput**, **low-latency**, **low cost**, and **high resource efficiency**
 - ❖ However, realizing these properties requires a series of **techniques & optimizations**

My Info



<https://github.com/Scusemua>



benrcarver@gmail.com



<https://scusemua.github.io/>

λFS CONCLUSION

